

Benemérita Universidad Autónoma de Puebla

Facultad de Ciencias de la Computación



T E S I S

**“Predicción de congestión en redes de computadoras
utilizando técnicas de Machine Learning”**

Presenta:

Ricardo Valera Martínez

Para obtener el grado de:

Licenciatura en Ingeniería en Ciencias de la Computación

Director: **Mauro Alberto López Muñoz**

Codirector: **Guillermo Colorado Jiménez**

Puebla, Pue., Febrero 2026

Índice general

1. Introducción	6
1.1. Planteamiento del problema	6
1.2. Objetivos	7
1.2.1. Objetivo General	8
1.2.2. Objetivos Específicos	8
1.3. Justificación	8
1.4. Alcances y Limitaciones	9
2. Marco Teórico	11
2.1. Fundamentos de redes de computadoras	11
2.1.1. Modelo OSI y TCP/IP	13
2.1.2. Concepto de congestión	14
2.1.3. Métricas de rendimiento	17
2.2. Técnicas de Machine Learning aplicables	20
2.2.1. Aprendizaje supervisado: regresión vs clasificación	20
2.2.2. Modelos de series temporales (ARIMA)	21
2.2.3. Modelos de aprendizaje automático (Random Forest, SVR)	22
2.2.4. Modelos de Deep Learning (LSTM, GRU)	23
2.2.5. Métricas de evaluación	23
3. Estado del Arte	26
3.1. Revisión de trabajos sobre predicción de tráfico con métodos estadísticos	27
3.2. Predicción de tráfico/congestión con Machine Learning clásico	28
3.3. Predicción con Deep Learning	29
3.4. Herramientas de monitoreo y datasets utilizados	31
3.5. Síntesis del estado del arte y brecha a cubrir	34
4. Metodología	40
4.1. Tipo de investigación	40
4.2. Definición operativa de congestión	42
4.3. Descripción de las métricas a utilizar	43
4.4. Fuente de datos	46
4.5. Preprocesamiento de datos	48
4.6. Modelos a implementar y justificación de su elección	52
4.7. Métricas de evaluación del modelo	55

4.8. Herramientas y entorno	58
5. Desarrollo e Implementación	61
5.1. Obtención y exploración de los conjuntos de datos	61
5.2. Preprocesamiento	65
5.3. Construcción del modelo ARIMA	66
5.4. Construcción del modelo Random Forest	68
5.5. Construcción del modelo LSTM	71
6. Resultados y Discusión	75
6.1. Resultados de ARIMA	75
6.2. Resultados de Random Forest	75
6.3. Resultados de LSTM	75
6.4. Comparativa de modelos	75
6.5. Discusión	75
7. Conclusiones	76
7.1. Cumplimiento de objetivos	76
7.2. Contribuciones	76
7.3. Limitaciones del estudio	76
Referencias bibliográficas	77
A. Código fuente de ejemplos críticos	80
B. Capturas de pantalla adicionales	81

Índice de figuras

2.1. Representación de una red entre dos computadoras.	11
2.2. Capas del modelo OSI.	14
2.3. Ubicación del problema de predicción de congestión dentro de los paradigmas de Machine Learning.	21
3.1. Taxonomía de las técnicas de monitoreo de red empleadas en la literatura revisada.	32
3.2. Línea de tiempo de la evolución metodológica en predicción de tráfico/congestión de red.	36
4.1. Pipeline de transformación de métricas crudas en variables de entrada y variable objetivo.	46
4.2. Construcción de secuencias de entrada mediante ventana deslizante para los modelos LSTM/GRU.	51
4.3. División cronológica del dataset primario en conjuntos de entrenamiento, validación y prueba.	51
4.4. Diseño experimental general: entrenamiento sobre el dataset primario y validación cruzada de fuentes contra MAWI.	55
4.5. Matriz de confusión para la tarea de clasificación de congestión.	56
5.1. Distribución del factor de utilización, dataset piloto MAWI.	64
5.2. ARIMA(1,1,1): predicción continua vs. valor real, conjunto de prueba.	68
5.3. Random Forest: predicción continua vs. valor real (escala normalizada), conjunto de prueba.	70
5.4. Importancia relativa de variables (Impureza de Gini), Random Forest.	71
5.5. Curva de aprendizaje de la red LSTM (pérdida MSE por época).	72
5.6. LSTM: predicción continua vs. valor real, conjunto de prueba.	73

Índice de cuadros

2.1. Resumen de métricas de rendimiento de red utilizadas en esta tesis.	20
2.2. Resumen comparativo de los modelos de predicción considerados en esta tesis.	25
3.1. Comparación de herramientas de monitoreo empleadas en la literatura revisada.	32
3.2. Comparación de datasets empleados en la literatura revisada frente a los datasets adoptados en esta tesis.	33
3.3. Síntesis comparativa de los trabajos revisados en el Estado del Arte.	35
3.4. Posicionamiento metodológico de esta tesis frente a la literatura revisada. . .	38
4.1. Clasificación del tipo de investigación de esta tesis.	42
4.2. Variables de entrada consideradas para los modelos predictivos.	44
4.3. Disponibilidad y método de obtención de métricas según la fuente de datos. .	45
4.4. Contraste del proceso de resamplio entre las dos fuentes de datos empleadas.	50
4.5. Resumen de las etapas de preprocesamiento de datos.	52
4.6. Modelos considerados durante la planeación de esta tesis y justificación de su exclusión del alcance experimental.	54
4.7. Rol de cada métrica de regresión en el análisis comparativo de esta tesis. . .	56
4.8. Síntesis del esquema de evaluación aplicado a los modelos de esta tesis. . . .	58
4.9. Herramientas empleadas para la recolección del dataset primario.	59
4.10. Bibliotecas de Python empleadas para la implementación de cada etapa metodológica.	59
5.1. Resultados de la extracción de telemetría sobre las trazas MAWI procesadas.	63
5.2. Estadísticas descriptivas del throughput y utilización derivada, dataset piloto MAWI (1,800 observaciones, granularidad de 1 minuto).	64
5.3. Composición de la división cronológica del dataset piloto MAWI tras el preprocesamiento.	65
5.4. Órdenes ARIMA con menor AIC, dataset piloto MAWI.	67
5.5. Métricas de desempeño en regresión de ARIMA(1,1,1), dataset piloto MAWI.	67
5.6. Desempeño base de clasificación y alertamiento (TAE) de ARIMA.	68
5.7. Métricas de regresión de Random Forest (escala normalizada), dataset MAWI.	69
5.8. Desempeño de clasificación y alertamiento (TAE) de Random Forest.	70
5.9. Arquitectura algorítmica de la red neuronal recurrente.	72
5.10. Métricas de regresión de LSTM (escala normalizada), dataset piloto MAWI.	73
5.11. Desempeño de clasificación temprana y alertamiento (TAE) de la red LSTM.	73
5.12. Validación de software: Evolución operativa de los tres paradigmas evaluados.	74

DEDICATORIA

*A mi padre, **Ricardo Valera Hernández**, por ser el pilar constante en cada etapa de mi formación, por su paciencia, su apoyo incondicional y por nunca dejar de creer en mí incluso cuando yo dudaba, por su ejemplo de responsabilidad y esfuerzo, por enseñarme que el trabajo honesto y la perseverancia son el camino para alcanzar cualquier meta.*

*A mi madre, **María Margarita Irma Martínez Pérez**, por ser la guía y el apoyo incondicional que hizo posible este trayecto. Gracias por su confianza infinita, por sus palabras que siempre supieron darme fuerza y por estar presente en cada paso de mi formación profesional. Este trabajo es testimonio de su esfuerzo y dedicación constante en mi vida.*

*A mi novia, **Vanessa F. Linarte Castañeda**, por su amor, su comprensión y su compañía en los momentos de cansancio y dificultad, por motivarme a seguir adelante cuando parecía imposible y por ser parte fundamental de este logro.*

*A mi hermano menor, **Emilio Valera Martínez**, por su complicidad y apoyo en los momentos de distracción y de enfoque. Gracias por estar ahí, por los consejos y por ser un compañero constante en este camino.*

*Al **Sr. Héctor H. Linarte** y la **Sra. Cristina E. Castañeda**, por recibirme en su familia como a un hijo más, por sus palabras de aliento y por el apoyo incondicional brindado durante este proceso. Su confianza y cariño fueron fundamentales para mantener el ánimo en la recta final.*

*A mi director de tesis, el **Mtro. Mauro Alberto López Muñoz**, y a mi codirector, el **Mtro. Guillermo Colorado Jiménez**, por su invaluable guía, su paciencia y por compartir su conocimiento para llevar este proyecto a buen puerto.*

Capítulo 1

Introducción

“El futuro de las redes de computadoras no solo depende de transmitir datos más rápido, sino de hacerlo de manera inteligente.”

La evolución en las redes de computadoras ha permitido el desarrollo de una gran cantidad de servicios y aplicaciones que forman parte de la vida cotidiana, desde la navegación web y las plataformas de comunicación, hasta los sistemas distribuidos, la computación en la nube y el intercambio masivo de información en tiempo real. Debido a este crecimiento constante en la cantidad de dispositivos conectados y el volumen de tráfico generado, las redes modernas enfrentan cada vez mayores desafíos relacionados con el rendimiento, la disponibilidad y la calidad del servicio.

1.1. Planteamiento del problema

Vivimos en una era digital en la que la conectividad se ha convertido en un servicio esencial, tan fundamental como la electricidad o el agua corriente. Desde las videollamadas que nos acercan a nuestros familiares y colegas, hasta las plataformas de streaming que nos ofrecen entretenimiento bajo demanda, pasando por la explosión del internet de las cosas (IoT) que llena nuestros hogares y ciudades de sensores inteligentes, gran parte de nuestra vida cotidiana depende de un flujo constante e ininterrumpido de datos. Según el informe *Cisco Annual Internet Report (2018–2023)*, el número de usuarios de Internet a nivel mundial pasó de 3900 millones en 2018 a 5300 millones en 2023, mientras que los dispositivos conectados a redes IP aumentaron de 18 400 millones a 29 300 millones en el mismo período [1].

Las redes de computadoras son el tejido que sostiene cambios inminentes en la industria. Sin embargo, no son un recurso infinito: cada enrutador, cada conmutador y cada enlace de fibra óptica tiene una capacidad limitada. Cuando la demanda de transmisión de datos supera esa capacidad se produce un fenómeno conocido como congestión de red. En términos sencillos (más adelante se abordará una definición apropiada para este trabajo), la congestión es el equivalente digital a un embotellamiento en una autopista: demasiados coches (paquetes de datos) intentando circular al mismo tiempo por un carril con aforo máximo. Las consecuencias se vuelven inmediatas y totalmente perceptibles para el usuario final: videollamadas

que se congelan, páginas web que cargan con lentitud, archivos que no terminan de descargarse o, en el peor de los casos, se pierde totalmente la conexión. En entornos críticos como hospitales, sistemas de redes financieras o redes industriales, estos fallos se traducen en pérdidas económicas millonarias o incluso riesgos para la seguridad de las personas.

Tradicionalmente, la gestión de redes ha tenido un enfoque reactivo: los administradores monitorizan equipos y, cuando detectan que un enlace está saturado, toman medidas correctivas como redirigir el tráfico, limitar ciertas aplicaciones o aumentar la capacidad del enlace. Este modo de actuar, aunque ha resultado funcional durante décadas, presenta una limitación fundamental: actúa después de que el problema ya ha comenzado a degradar la calidad del servicio. Pensemos en un coche autónomo que necesita tomar decisiones en milisegundos en tiempo real o en un cirujano operando a distancia mediante un robot; reaccionar después ya no es una opción, se necesita anticipación.

De lo anterior se desprende el problema central que da origen a esta investigación: los mecanismos actuales de gestión de redes de computadoras carecen, en su mayoría, de la capacidad de anticipar la congestión antes de que esta se manifieste, limitándose a detectarla y corregirla una vez que ya ha comenzado a degradar la calidad del servicio. Esta naturaleza reactiva impide a los administradores de red y a los sistemas automatizados de gestión tomar decisiones oportunas —como redirigir tráfico, priorizar aplicaciones críticas o escalar recursos— antes de que el impacto sea perceptible para el usuario final. El problema de investigación puede formularse, entonces, de la siguiente manera: **¿es posible, a partir del análisis de datos históricos de operación de una red, construir un modelo capaz de predecir con suficiente antelación que un enlace alcanzará un estado de congestión, de tal forma que dicha predicción resulte útil para la toma de decisiones antes de que la degradación del servicio se materialice?**

En este contexto, y una vez planteado formalmente el problema, las técnicas de Machine Learning (ML) se presentan como una vía prometedora para abordarlo. El ML, una rama de la inteligencia artificial, permite a las máquinas aprender patrones a partir de datos históricos y realizar predicciones sin ser programadas explícitamente para cada escenario, lo que lo convierte en una herramienta natural para atacar el problema de anticipación recién descrito.

El presente trabajo de tesis se sitúa justo en esta intersección entre las redes de computadoras y la inteligencia artificial. El objetivo central es desarrollar un sistema basado en técnicas de Machine Learning que sea capaz de predecir la congestión en un enlace de red a partir del análisis de métricas de tráfico obtenidas de forma híbrida activa/pasiva. En lugar de limitarnos a reaccionar cuando la utilización ya está en niveles críticos, busco construir un modelo que pueda alertar con varios minutos de antelación: “Atención: este enlace alcanzará el 85 % de su capacidad; toma medidas ahora”.

1.2. Objetivos

A continuación se detallan los objetivos que guían el presente trabajo de tesis, los cuales fueron definidos en el protocolo de investigación y constituyen la hoja de ruta para el desarrollo del sistema propuesto.

1.2.1. Objetivo General

El objetivo general es desarrollar un sistema basado en Machine Learning que permita predecir la congestión de redes de computadoras a partir del análisis de datos de operación, evaluando su desempeño en escenarios de tráfico de distinta escala y naturaleza, con el fin de aportar evidencia sobre la viabilidad de este enfoque predictivo para optimizar la gestión de recursos y la calidad del servicio en redes de computadoras.

1.2.2. Objetivos Específicos

- Recopilar y organizar datos de operación de una red de computadoras, incluyendo parámetros como frecuencia de operación, ancho de banda, rendimiento y disponibilidad.
- Diseñar un procedimiento para almacenar, depurar y procesar los datos obtenidos de una manera estructurada.
- Implementar técnicas de Machine Learning adecuadas para analizar los patrones de tráfico y comportamiento de la red.
- Desarrollar un prototipo que realice predicciones de posibles escenarios de congestión de red.
- Validar el desempeño del sistema mediante métricas de evaluación.
- Identificar posibles aplicaciones reales del sistema propuesto en distintos contextos de uso.

1.3. Justificación

El crecimiento exponencial del tráfico en las redes de computadoras ha superado las estrategias tradicionales de gestión basadas en la monitorización reactiva. Hoy en día, la demanda de ancho de banda generada por servicios de videoconferencia, plataformas de streaming, aplicaciones en la nube y dispositivos del Internet de las Cosas somete a las infraestructuras a picos de utilización impredecibles. En este escenario, los administradores se ven forzados a intervenir únicamente después de que los niveles de congestión ya han degradado la calidad del servicio, lo que deriva en una experiencia deficiente para los usuarios y, en ámbitos críticos, en pérdidas económicas o riesgos de seguridad. Por ello, se vuelve indispensable desarrollar herramientas que permitan anticipar la saturación de los enlaces, migrando de una operación reactiva a una gestión predictiva que tome decisiones antes de que la congestión se materialice.

La aplicación de técnicas de Machine Learning en el ámbito de las redes abre una oportunidad real para este cambio de paradigma [2]. A diferencia de los métodos estadísticos convencionales, los algoritmos de aprendizaje automático tienen la capacidad de descubrir patrones complejos y no lineales a partir de grandes volúmenes de datos de operación, como series temporales de utilización, latencia o pérdida de paquetes. Modelos como las redes neuronales recurrentes, los bosques aleatorios o las arquitecturas de aprendizaje profundo han

demostrado un desempeño notable en la predicción de tráfico en otros contextos, pero su aplicación sistemática para anticipar la congestión en enlaces individuales constituye un área de investigación con vacíos aún por cubrir, particularmente en lo referente a la comparación de múltiples familias de modelos en condiciones realistas de operación.

En el ámbito práctico, un sistema que prediga con antelación suficiente que un enlace alcanzará su capacidad límite puede transformar la administración de redes. Por ejemplo, un operador podría redirigir flujos de tráfico, activar mecanismos de balance de carga, reducir la prioridad de aplicaciones no críticas o incluso aprovisionar recursos adicionales de forma automática antes de que el usuario note degradación alguna. Esta capacidad de anticipación resulta crucial en entornos donde la continuidad del servicio es prioritaria, como hospitales que dependen de telemedicina, entidades financieras con transacciones en tiempo real o plantas industriales con procesos automatizados. De esta manera, la presente investigación no solo busca generar conocimiento teórico, sino también ofrecer una herramienta con un impacto directo en la fiabilidad y eficiencia de las infraestructuras digitales.

Desde el punto de vista metodológico, la propuesta de construir un prototipo basado en datos de operación reales y de evaluarlo mediante métricas rigurosas —como el error cuadrático medio, el error absoluto medio y la comparación de tasas de predicción a distintos horizontes temporales— aporta un procedimiento replicable para la comunidad académica y profesional. La selección de tres paradigmas de modelado (series temporales estadísticas, aprendizaje automático clásico y aprendizaje profundo) permite además trazar un mapa claro de las fortalezas y limitaciones de cada enfoque, ofreciendo criterios objetivos para futuras implementaciones. En un campo en el que la mayoría de los estudios se concentran en la predicción general del volumen de tráfico, esta tesis se distingue por colocar el foco en la detección temprana de un evento específico y operacionalmente relevante: la congestión.

Finalmente, la investigación se justifica por su alineación con las tendencias actuales de la industria hacia las redes autónomas y la inteligencia artificial aplicada a las operaciones de TI (AIOps). Grandes empresas del sector tecnológico y organismos de estandarización están impulsando la incorporación de modelos predictivos en los sistemas de gestión de red, reconociendo que la simple supervisión ya no es suficiente para soportar las demandas de latencia y disponibilidad de las aplicaciones del futuro. En este sentido, el trabajo propuesto contribuye a cerrar la brecha entre las capacidades teóricas del Machine Learning y su aplicación práctica en un problema real de ingeniería de redes, generando evidencia empírica sobre su viabilidad, precisión y pertinencia en escenarios de congestión.

1.4. Alcances y Limitaciones

El presente trabajo de tesis se circunscribe al desarrollo de un prototipo funcional de predicción de congestión, implementado enteramente en código y orientado a validar la viabilidad sobre un conjunto de datos de operación de red. El alcance del proyecto se limita a la construcción, entrenamiento y evaluación de estos modelos mediante métricas cuantitativas, sin comprometerse con el desarrollo de una interfaz gráfica de usuario, una aplicación empaquetada o un sistema de producción completo. Por lo tanto, los resultados deben interpretarse como una prueba de concepto que demuestra la capacidad predictiva de los algoritmos seleccionados, no como una solución lista para ser desplegada en un entorno operacional real.

En términos de datos, el prototipo se desarrolla y evalúa sobre dos fuentes complementarias: una red privada de escala reducida sobre la cual se realizó recolección propia de datos de operación durante un periodo de NUMEROFINAL días, y un archivo académico de tráfico de Internet de gran escala empleado como conjunto de validación externa. Si bien esta combinación permite evaluar el desempeño de los modelos bajo condiciones de tráfico de naturaleza y escala distintas, ampliando el alcance de las conclusiones respecto de un estudio basado en una única fuente, no debe interpretarse como una validación exhaustiva sobre la totalidad de topologías, tecnologías de acceso o patrones de tráfico posibles en una red de computadoras. La generalización de los resultados a escenarios sustancialmente distintos a los aquí evaluados —por ejemplo, redes móviles, entornos industriales o infraestructuras de centros de datos— queda fuera del alcance de este trabajo y se plantea como una línea de investigación futura.

Desde el punto de vista computacional, se ha establecido un límite práctico en la complejidad de los algoritmos y en los recursos de hardware empleados. El prototipo se ejecuta en un entorno de estación de trabajo convencional, sin recurrir a infraestructura de cómputo distribuido o aceleración especializada (GPU de alto rendimiento). En consecuencia, los modelos profundos se configuran con un número acotado de capas y neuronas, y no se explora el ajuste exhaustivo de hiperparámetros mediante búsqueda en malla de gran tamaño. Este enfoque permite obtener resultados en tiempos razonables, pero sacrifica potencialmente la obtención del modelo óptimo absoluto, lo cual es una limitación deliberada acorde con el carácter prototípico del desarrollo.

Otra restricción relevante es que el sistema no opera en tiempo real sobre una red viva, sino que entrena y evalúa sobre trazas históricas. La predicción se realiza fuera de línea, asumiendo que los datos han sido previamente recolectados y preprocesados. Por tanto, no se han implementado mecanismos de ingestión continua de datos, detección de cambio de concepto ni reentrenamiento automático del modelo, aspectos que resultarían necesarios para una eventual transición a un sistema de alerta temprana en producción. Esta limitación es congruente con la elección de priorizar la comparación de modelos bajo condiciones controladas [3].

Finalmente, es imperativo distinguir entre la variable objetivo del sistema y sus variables predictoras. Para fines de etiquetado y evaluación, el estado de congestión se define y cuantifica exclusivamente a partir del factor de utilización del enlace (variable objetivo). Sin embargo, el prototipo predictivo sí incorpora métricas de red complementarias —como estimaciones de rendimiento y disponibilidad obtenidas de forma pasiva— como variables de entrada (*features*) para el entrenamiento de los algoritmos. Esta delimitación conceptual permite definir un umbral matemático claro para la congestión, reconociendo simultáneamente que el modelo requiere de una perspectiva multivariable para capturar los patrones complejos que preceden a la saturación del enlace.

Capítulo 2

Marco Teórico

2.1. Fundamentos de redes de computadoras

Imaginemos por un momento dos computadoras aisladas, cada una con información valiosa almacenada en su interior. Para que esos datos puedan ser aprovechados por el otro equipo, alguien debe copiarlos en un dispositivo de almacenamiento externo, trasladarlos físicamente e introducirlos en la segunda máquina. Este procedimiento, aunque funcional, es lento, propenso a errores humanos y completamente inviable cuando la cantidad de información crece o cuando quien la necesita se encuentra a cientos o miles de kilómetros de distancia. Precisamente para resolver esta limitación nacieron las redes de computadoras: sistemas que conectan dos o más dispositivos de cómputo mediante algún medio de transmisión —cables de cobre, fibra óptica o el espectro electromagnético— con el propósito de intercambiar datos de manera rápida, confiable y eficiente [4].

En esencia, una red de computadoras es un conjunto de nodos autónomos interconectados que cooperan para compartir recursos e información. El término "nodo" designa a cualquier dispositivo capaz de originar, recibir o retransmitir datos: computadoras personales, servidores, teléfonos móviles, sensores, impresoras e incluso electrodomésticos inteligentes. Estos nodos se comunican a través de enlaces, que son los caminos físicos o inalámbricos por los que viaja la información. A su vez, los enlaces se conectan entre sí por medio de dispositivos de interconexión como conmutadores y enrutadores, cuya misión es encaminar los datos desde su origen hasta su destino, eligiendo la ruta más adecuada en cada momento y gestionando las colas de espera que se forman cuando varios flujos de tráfico coinciden en un mismo punto. El comportamiento de estas colas de espera bajo condiciones de alta demanda es, como se detallará en la sección 2.1.2, el origen mismo del fenómeno de congestión.

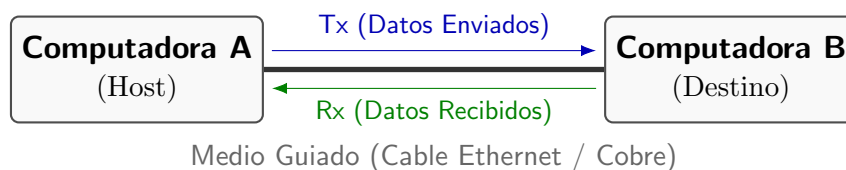


Figura 2.1: Representación de una red entre dos computadoras.

El funcionamiento de una red se organiza de manera jerárquica para domar la complejidad inherente a la comunicación digital. Esta organización se plasma en los llamados modelos de referencia, que dividen el problema de transmitir información en varias capas superpuestas, cada una con responsabilidades claramente acotadas: desde los aspectos puramente físicos de la transmisión (voltajes, pulsos de luz, ondas de radio) hasta la interacción con las aplicaciones que el usuario ve en pantalla. Esta arquitectura en capas permite que un fabricante mejore un componente sin necesidad de rediseñar toda la red, y que tecnologías muy dispares puedan interoperar siempre que respeten los protocolos definidos en cada nivel. El modelo de referencia más utilizado para describir esta organización, el modelo OSI, se desarrolla con mayor profundidad en la sección 2.1.1.

Más allá de cómo se organizan las capas, existe una decisión de diseño fundamental que determina cómo viajan realmente los datos a través de una red: el paradigma de conmutación. En las redes telefónicas tradicionales se empleaba la **conmutación de circuitos**: cuando dos personas iniciaban una llamada, la red reservaba un camino físico dedicado y exclusivo entre ambos extremos durante toda la conversación. Este enfoque garantiza una calidad de servicio constante y predecible, pero resulta profundamente ineficiente desde el punto de vista de la utilización de recursos: si los interlocutores hacen una pausa de diez segundos, el circuito reservado permanece completamente ocioso y ningún otro usuario puede aprovecharlo, incluso si la red entera está saturada de solicitudes en ese instante.

Las redes de computadoras modernas, incluida Internet, optaron por un modelo radicalmente distinto: la **conmutación de paquetes**. En este paradigma, los datos se fragmentan en pequeñas unidades llamadas paquetes o datagramas, cada una encabezada por información de control (dirección de origen, destino, número de secuencia, entre otros campos). Cada paquete viaja de nodo en nodo de manera independiente y sin reservar recursos exclusivos, compartiendo dinámicamente los enlaces con paquetes pertenecientes a otras comunicaciones completamente distintas. Cuando un remitente hace una pausa en su transmisión, otros flujos aprovechan inmediatamente el enlace liberado, lo que multiplica la eficiencia global del sistema comparado con la conmutación de circuitos [5].

Esta compartición estadística de los recursos es, precisamente, la razón por la que Internet puede ofrecer conectividad a miles de millones de dispositivos con una infraestructura física finita. Sin embargo, tiene una contrapartida inevitable: como ningún flujo individual tiene garantizado un ancho de banda exclusivo, si demasiados remitentes transmiten de manera simultánea a través del mismo enlace, la demanda agregada puede superar la capacidad disponible. Es precisamente en este punto —cuando la suma de los flujos que compiten por un recurso compartido excede lo que ese recurso puede entregar— donde se origina el fenómeno que esta tesis busca predecir: la congestión de red, que se define y caracteriza formalmente en la siguiente sección.

Capacidad del enlace: ancho de banda, throughput y utilización

Para poder cuantificar cuándo un enlace se encuentra cerca de su límite, es necesario introducir tres métricas básicas que se emplearán a lo largo de toda la tesis. Cada enlace en una red posee una capacidad máxima teórica de transmisión, denominada **ancho de banda** (*bandwidth*), que se mide en bits por segundo (bps). Un enlace de fibra óptica típico puede ofrecer 1 Gbps, mientras que una conexión inalámbrica doméstica quizás solo alcance 50

Mbps. Esta cifra representa el límite superior teórico del enlace, no la cantidad de datos que efectivamente circula por él en un momento dado.

El tráfico real que atraviesa el enlace en un instante determinado se conoce como **throughput** o rendimiento efectivo, y su valor siempre es inferior o igual al ancho de banda nominal, ya que factores como la sobrecarga de los encabezados de protocolo, las retransmisiones y el propio comportamiento del tráfico impiden alcanzar el máximo teórico de manera sostenida. La razón entre el throughput observado y el ancho de banda disponible se denomina **factor de utilización**, y constituye una de las métricas más reveladoras sobre el estado de salud de un enlace:

$$U = \frac{\text{Throughput observado}}{\text{Ancho de banda nominal}} \quad (2.1)$$

Mientras el factor de utilización se mantenga en niveles moderados, la red opera con holgura: existe margen suficiente para absorber ráfagas de tráfico sin que se produzcan retrasos perceptibles. Conforme U se aproxima a 1 (es decir, al 100 % de la capacidad), comienzan a manifestarse los síntomas característicos de la congestión, los cuales se explican en detalle a continuación.

2.1.1. Modelo OSI y TCP/IP

El Modelo de Interconexión de Sistemas Abiertos (OSI, *Open Systems Interconnection*), estandarizado por la Organización Internacional de Normalización (ISO) en 1984, constituye un marco conceptual que divide el proceso de comunicación en redes en siete capas funcionales independientes [4]. Cada nivel abstrae un conjunto específico de responsabilidades, desde la transmisión física de bits hasta la interacción con las aplicaciones del usuario. Esta estratificación facilita la interoperabilidad entre equipos de distintos fabricantes y permite analizar los fenómenos de red, como la congestión, dentro de una estructura bien definida. En particular, la gestión de la congestión incide directamente en las capas de transporte y red, donde se regulan el control de flujo, el enrutamiento y la asignación de recursos, aspectos que serán modelados en este trabajo a partir de métricas de utilización.



Figura 2.2: Capas del modelo OSI.

2.1.2. Concepto de congestión

Con las nociones de ancho de banda, throughput y utilización ya establecidas, es posible ahora caracterizar con precisión qué ocurre dentro de una red cuando la demanda de tráfico se aproxima o supera la capacidad disponible de un enlace. A este fenómeno se le denomina **congestión de red**, y constituye el objeto central de estudio de esta tesis.

Qué ocurre dentro de un enrutador

Para entender el origen físico de la congestión es necesario examinar qué sucede dentro de un enrutador o conmutador cuando procesa tráfico. Cada dispositivo de interconexión dispone de pequeñas memorias temporales llamadas **buffers**, donde almacena los paquetes que van llegando mientras esperan ser procesados y reenviados por el enlace de salida correspondiente. Si los paquetes llegan a una tasa inferior a la capacidad del enlace de salida, el buffer apenas se ocupa y los paquetes son despachados casi de inmediato, sin apenas tiempo de espera.

El problema surge cuando, durante un intervalo sostenido, la tasa de llegada de paquetes supera la tasa a la que el enlace de salida puede transmitirlos. En ese escenario, la cola de espera dentro del buffer comienza a crecer progresivamente. Este fenómeno es análogo a lo que ocurre en una caseta de cobro de autopista: mientras los vehículos lleguen a un ritmo menor al que la caseta puede atender, la fila permanece corta y el retraso es mínimo; pero si llegan más vehículos de los que la caseta puede procesar, la fila se alarga sin control y el tiempo de espera se dispara de forma no lineal. Es importante subrayar este último punto: el retraso generado por la congestión **no crece de manera proporcional** al exceso de tráfico, sino que tiende a dispararse de forma abrupta una vez que se cruza cierto umbral de utilización, un comportamiento bien documentado en la teoría de colas aplicada a redes de datos [6].

Latencia y sus componentes

El tiempo total que un paquete tarda en viajar desde que ingresa a la red hasta que llega a su destino se conoce como **latencia** o retardo extremo a extremo, y se compone de cuatro elementos:

- **Tiempo de propagación:** lo que tarda la señal física en recorrer el medio de transmisión (cable, fibra, aire), determinado por la distancia y la velocidad de propagación de la señal.
- **Tiempo de transmisión:** lo que tarda el nodo emisor en colocar todos los bits del paquete sobre el medio físico, el cual depende del tamaño del paquete y del ancho de banda del enlace.
- **Tiempo de procesamiento:** el que cada nodo intermedio invierte en examinar el encabezado del paquete y decidir hacia dónde reenviarlo.
- **Tiempo de encolado:** el tiempo que el paquete permanece esperando dentro del buffer de un enrutador antes de ser transmitido.

En condiciones normales de operación, el tiempo de encolado es prácticamente insignificante frente a los otros tres componentes. Sin embargo, bajo congestión, este componente puede convertirse —y de hecho suele convertirse— en el factor dominante de la latencia total, siendo responsable de la gran mayoría del retraso que experimenta el usuario final.

Jitter: la variabilidad del retraso

Dado que la llegada de paquetes a un enrutador es irregular e impredecible por naturaleza, el tamaño de la cola de espera fluctúa constantemente a lo largo del tiempo. Como consecuencia, la latencia experimentada por paquetes consecutivos de un mismo flujo no es constante, sino que varía de un paquete a otro. Esta variabilidad en el retardo se denomina **jitter**, y resulta particularmente perjudicial para aplicaciones sensibles al tiempo que requieren una entrega de datos a ritmo constante, como las videollamadas, los juegos en línea o la transmisión de audio y video en tiempo real, donde un jitter elevado se traduce en cortes, distorsión o congelamiento de la imagen incluso cuando la latencia promedio parece aceptable.

Pérdida de paquetes y la respuesta reactiva de TCP

Cuando la congestión se agrava lo suficiente, el buffer de un enrutador puede llegar a llenarse por completo. Llegado ese punto, el dispositivo no tiene más opción que descartar los paquetes entrantes que ya no caben en la memoria disponible, un evento conocido como **pérdida de paquetes**. Una tasa de pérdida ocasional puede ser tolerable e incluso pasar desapercibida para el usuario, pero cuando se vuelve frecuente, degrada severamente la calidad de la comunicación.

Es en este punto donde interviene el comportamiento de los protocolos de transporte, particularmente TCP. Este protocolo interpreta la pérdida de paquetes como una señal indirecta de que la red está congestionada, y en respuesta reduce drásticamente su tasa de

envío mediante mecanismos de control de congestión [7], para posteriormente incrementarla de manera gradual mientras no detecte nuevas pérdidas. Este mecanismo, formalizado en estándares como el RFC 2914 [8], ha sido fundamental para preservar la estabilidad de Internet durante décadas. No obstante, es crucial notar su naturaleza intrínsecamente **reactiva**: el mecanismo solo se activa *después* de que la congestión ya ha provocado pérdidas reales, momento en el cual la calidad del servicio ya se ha degradado y el usuario final ya ha percibido el problema. Esta limitación reactiva es, precisamente, la motivación central que da origen al enfoque predictivo propuesto en esta tesis.

El cuello de botella

En cualquier ruta que conecte dos nodos de una red, típicamente a través de múltiples enlaces intermedios, existe un concepto adicional relevante: el **cuello de botella** (*bottleneck*). Se trata del enlace con menor capacidad disponible a lo largo de toda la ruta, y es este el que determina la velocidad máxima a la que pueden fluir los datos de extremo a extremo, de manera análoga a como el tramo más estrecho de una tubería limita el caudal total de agua que puede circular por ella, sin importar qué tan anchos sean los demás tramos. Los cuellos de botella son, por definición, los puntos donde la congestión se manifiesta primero y con mayor intensidad dentro de una red, y por lo tanto son los candidatos naturales sobre los cuales aplicar técnicas de predicción y prevención, como la que se propone en este trabajo.

Definición operativa de congestión para esta tesis

A partir de los elementos anteriores —utilización, buffers, latencia, jitter, pérdida de paquetes y cuellos de botella— es posible ahora formular una definición precisa y, sobre todo, *medible* de congestión, que es la que se adoptará de manera consistente a lo largo de todo el desarrollo metodológico y experimental de esta tesis.

Se dice que un enlace se encuentra en estado de **congestión** cuando su factor de utilización U (Ecuación 2.1) supera un umbral crítico U_{th} durante un intervalo de tiempo sostenido, más allá del cual el comportamiento del sistema deja de ser lineal y comienzan a manifestarse de forma perceptible los síntomas descritos anteriormente: crecimiento acelerado del tiempo de encolado, incremento del jitter y, en los casos más severos, pérdida de paquetes. Formalmente:

$$\text{Congestión} \iff U(t) \geq U_{th} \quad \text{durante un intervalo } [t, t + \Delta t] \quad (2.2)$$

La determinación del umbral crítico U_{th} y la ventana de observación Δt no se establecerá de manera arbitraria. Para garantizar el rigor del etiquetado, el valor de U_{th} se derivará empíricamente en el Capítulo 4 a partir de un análisis estadístico sobre el conjunto de datos de operación histórica. Específicamente, se buscará identificar el punto de inflexión en la distribución de los datos (por ejemplo, mediante el percentil 85 o 90) donde la métrica de utilización comienza a correlacionarse con un incremento exponencial en los retardos o fluctuaciones de la red. De igual forma, el intervalo Δt estará dictado por la granularidad de muestreo soportada por las herramientas de instrumentación pasiva empleadas durante la recolección de las trazas.

Esta definición operativa presenta dos ventajas fundamentales para los fines de esta investigación. En primer lugar, se apoya en una métrica directamente observable y de fácil

recolección de forma pasiva —el factor de utilización—, a diferencia de otras métricas como el jitter o la pérdida de paquetes, que requieren instrumentación más invasiva o cálculos derivados más complejos. En segundo lugar, al tratarse de un umbral cuantitativo y no de una apreciación cualitativa, permite convertir el problema de predecir congestión en un problema bien definido de aprendizaje automático: ya sea como una tarea de **regresión** (predecir el valor futuro de $U(t)$) o como una tarea de **clasificación binaria** (predecir si $U(t)$ superará o no el umbral U_{th} en un horizonte temporal determinado), ambos enfoques que se explorarán y compararán en los capítulos metodológico y experimental de este trabajo.

Es importante reconocer, sin embargo, que esta definición constituye una simplificación deliberada de un fenómeno inherentemente multifactorial. Como se mencionó en la sección de alcances y limitaciones, otras dimensiones de la congestión —como el jitter o la tasa de pérdida de paquetes— no son modeladas de forma directa por el prototipo desarrollado en esta tesis, aunque su comprensión teórica, expuesta en esta sección, resulta indispensable para justificar por qué la utilización constituye un proxy razonable y suficientemente informativo del estado de congestión de un enlace.

2.1.3. Métricas de rendimiento

En la sección anterior se explicó *qué* son la utilización, la latencia, el jitter y la pérdida de paquetes, y *por qué* aparecen como consecuencia de la congestión. Corresponde ahora precisar *cómo* se cuantifica formalmente cada una de estas métricas, qué unidades emplean y mediante qué mecanismos se obtienen en la práctica, ya que estas definiciones son las que se emplearán posteriormente para construir el conjunto de variables (*features*) que alimentarán los modelos de Machine Learning propuestos en esta tesis.

Utilización del enlace

Como se estableció en la Ecuación 2.1, la utilización U es la razón entre el throughput observado y el ancho de banda nominal del enlace. En términos de instrumentación, esta métrica se obtiene típicamente mediante el protocolo SNMP (*Simple Network Management Protocol*), consultando periódicamente los contadores de octetos entrantes y salientes de una interfaz de red (`ifInOctets` e `ifOutOctets` de la MIB-II) [4]. Dado que estos contadores son acumulativos, el throughput en un intervalo $[t_1, t_2]$ se calcula como la diferencia entre lecturas consecutivas dividida entre el tiempo transcurrido:

$$\text{Throughput}(t_1, t_2) = \frac{8 \times (\text{Octetos}(t_2) - \text{Octetos}(t_1))}{t_2 - t_1} \quad (2.3)$$

donde el factor 8 convierte de bytes a bits, ya que el ancho de banda se expresa convencionalmente en bits por segundo. La granularidad del muestreo (por ejemplo, cada 30 segundos, 1 minuto o 5 minutos) determina la resolución temporal con la que posteriormente se podrá detectar y predecir la congestión, y constituye una decisión metodológica que se abordará en el Capítulo 4.

Latencia (RTT)

La latencia de ida y vuelta, o *Round-Trip Time* (RTT), es el tiempo que transcurre entre el envío de un paquete y la recepción de su respuesta. Se mide de forma activa mediante herramientas como **ping** (que emplea paquetes ICMP *Echo Request/Reply*) o mediante el análisis de las marcas de tiempo (*timestamps*) del protocolo TCP durante el establecimiento de una conexión (*three-way handshake*) [5]. Formalmente:

$$\text{RTT} = t_{\text{recepción}} - t_{\text{envío}} \quad (2.4)$$

En la práctica no se trabaja con una sola medición de RTT, sino con una serie de mediciones sucesivas, ya que es precisamente la variabilidad entre estas mediciones consecutivas la que da origen al jitter, descrito a continuación.

Jitter

A diferencia de la explicación conceptual ofrecida en la sección anterior, aquí se presenta la definición formal adoptada por el estándar RFC 3550, que especifica el protocolo RTP (*Real-time Transport Protocol*) utilizado ampliamente en aplicaciones de voz y video sobre IP [9]. Dicho estándar define el jitter como la variación estadística en el tiempo de llegada entre paquetes consecutivos, y propone un estimador de primer orden que se actualiza de forma incremental con cada nuevo paquete recibido:

$$J(i) = J(i-1) + \frac{|D(i-1, i)| - J(i-1)}{16} \quad (2.5)$$

donde $D(i-1, i)$ representa la diferencia en los tiempos de tránsito relativos entre dos paquetes consecutivos $i-1$ e i , calculada como:

$$D(i-1, i) = (R_i - R_{i-1}) - (S_i - S_{i-1}) \quad (2.6)$$

siendo R_i el instante de llegada del paquete i en el receptor y S_i la marca de tiempo con la que dicho paquete fue enviado por el emisor. El factor $1/16$ actúa como una ganancia de suavizado que da lugar a un estimador con buen equilibrio entre estabilidad estadística y capacidad de reacción ante cambios súbitos, evitando que picos aislados distorsionen la medición general [9]. Esta es la definición de jitter que se adoptará como referencia en esta tesis en caso de emplearse como variable adicional de análisis.

Pérdida de paquetes

La tasa de pérdida de paquetes (*packet loss rate*) se expresa como el porcentaje de paquetes enviados que nunca llegan a su destino, respecto del total de paquetes transmitidos en un intervalo de observación:

$$P_{\text{loss}} = \frac{\text{Paquetes enviados} - \text{Paquetes recibidos}}{\text{Paquetes enviados}} \times 100 \% \quad (2.7)$$

Esta métrica puede obtenerse de dos maneras. La primera es de forma **activa**, inyectando tráfico de prueba (por ejemplo, mediante herramientas como **iperf** o series de solicitudes

ICMP) y contabilizando cuántas respuestas se reciben de vuelta. La segunda es de forma **pasiva**, consultando directamente los contadores de descarte que los propios dispositivos de red exponen a través de SNMP (como `ifInDiscards` o `ifOutDiscards`), sin necesidad de generar tráfico adicional que pudiera, irónicamente, contribuir a la propia congestión que se busca medir [4]. Esta segunda aproximación resulta preferible en el contexto de esta tesis, dado que el planteamiento del problema exige una recolección de datos estrictamente pasiva.

Throughput vs. Goodput

Conviene distinguir con precisión entre dos términos que suelen confundirse. El **throughput**, definido en la Ecuación 2.3, contabiliza la totalidad de los bits que atraviesan el enlace en un intervalo de tiempo, incluyendo los encabezados de los distintos protocolos (Ethernet, IP, TCP/UDP) y cualquier paquete retransmitido debido a pérdidas previas. El **goodput**, en cambio, contabiliza únicamente los bits de datos útiles a nivel de aplicación que fueron entregados exitosamente, excluyendo dicho overhead de protocolo:

$$\text{Goodput} = \frac{\text{Bits útiles entregados a la capa de aplicación}}{\text{Tiempo total de transmisión}} \quad (2.8)$$

Bajo condiciones de congestión severa, es posible observar un throughput aparentemente estable mientras el goodput cae drásticamente, ya que una proporción creciente del tráfico corresponde a retransmisiones de paquetes perdidos en lugar de datos nuevos. Esta distinción resulta relevante para esta tesis porque un modelo que solo observe throughput podría no detectar que la red ya se encuentra operando de manera ineficiente por efecto de la congestión.

Disponibilidad

Finalmente, la **disponibilidad** (*availability*) de un enlace o servicio de red se define como la proporción de tiempo durante la cual dicho recurso se encuentra operativo y accesible, respecto del tiempo total de observación:

$$\text{Disponibilidad} = \frac{\text{Tiempo operativo}}{\text{Tiempo operativo} + \text{Tiempo de inactividad}} \times 100 \% \quad (2.9)$$

Si bien la disponibilidad se asocia tradicionalmente con fallos de hardware o interrupciones del servicio más que con la congestión en sí misma, ambos fenómenos están relacionados: una congestión suficientemente severa y sostenida puede degradar la calidad del servicio hasta un punto en que, desde la perspectiva del usuario final, resulta indistinguible de una interrupción total. Por esta razón, y dado que fue contemplada explícitamente entre los objetivos específicos de este trabajo, la disponibilidad se incorpora como una métrica complementaria dentro del conjunto de datos de operación a recopilar.

Síntesis de métricas

La Tabla 2.1 sintetiza las métricas descritas en esta sección, sus unidades y su método de obtención, y constituye la base sobre la cual se construirá el conjunto de variables de entrada (*features*) del sistema de predicción propuesto en el Capítulo 4.

Métrica	Unidad	Ecuación	Método de obtención
Utilización	Adimensional (%)	2.1	SNMP (pasivo)
Throughput	bits/s	2.3	SNMP (pasivo)
Latencia (RTT)	ms	2.4	ICMP / TCP timestamps (activo)
Jitter	ms	2.5	RTCP / estimación RFC 3550
Pérdida de paquetes	%	2.7	SNMP (pasivo) / probes (activo)
Goodput	bits/s	2.8	Derivado de throughput
Disponibilidad	%	2.9	Monitoreo continuo

Cuadro 2.1: Resumen de métricas de rendimiento de red utilizadas en esta tesis.

2.2. Técnicas de Machine Learning aplicables

Una vez establecidos los fundamentos de redes de computadoras y caracterizado formalmente el fenómeno de la congestión, corresponde ahora presentar el conjunto de herramientas metodológicas que esta tesis emplea para abordar el problema de predicción planteado: las técnicas de Machine Learning (ML). Esta sección no pretende ser un tratado exhaustivo de aprendizaje automático, sino una revisión dirigida y suficiente de los conceptos, paradigmas y modelos específicos que resultan pertinentes para la tarea de predecir series temporales de utilización de red, y que serán empleados e implementados en los capítulos de Metodología (Capítulo 4) y Desarrollo (Capítulo 5).

2.2.1. Aprendizaje supervisado: regresión vs clasificación

El aprendizaje automático se organiza tradicionalmente en tres grandes paradigmas: el aprendizaje supervisado, en el que el modelo aprende a partir de ejemplos previamente etiquetados con la respuesta correcta; el aprendizaje no supervisado, en el que el modelo busca patrones o estructuras ocultas en datos sin etiquetar; y el aprendizaje por refuerzo, en el que un agente aprende a partir de la interacción con un entorno y de las recompensas obtenidas. El problema de predicción de congestión planteado en esta tesis se enmarca de manera natural dentro del **aprendizaje supervisado**, ya que se dispone de datos históricos de operación de red (las variables de entrada o *features*, como utilización, latencia o pérdida de paquetes en instantes pasados) junto con el valor real observado que se desea predecir (la utilización futura del enlace).

Dentro del aprendizaje supervisado, y de acuerdo con la naturaleza de la variable que se busca predecir, es posible distinguir dos tareas fundamentales:

- **Regresión:** la variable objetivo es continua. En el contexto de esta tesis, esto correspondería a predecir directamente el valor futuro del factor de utilización $U(t+h)$ para un horizonte de predicción h , expresado como un número real entre 0 y 1.
- **Clasificación:** la variable objetivo es discreta o categórica. En este caso, correspondería a predecir si el enlace se encontrará o no en estado de congestión en el horizonte h , de acuerdo con la definición operativa formulada en la Ecuación 2.2 —es decir, si $U(t+h) \geq U_{th}$ —, reduciendo el problema a una clasificación binaria (congestión / no congestión).

Ambos enfoques no son mutuamente excluyentes, sino complementarios: un modelo de regresión ofrece información más rica (el valor exacto esperado de utilización), mientras que uno de clasificación ofrece una respuesta más directamente accionable para un sistema de alertas (sí/no hay riesgo de congestión). Por esta razón, en esta tesis se explorará la formulación del problema bajo ambos enfoques según el modelo empleado, tal como se detalla en las siguientes subsecciones.

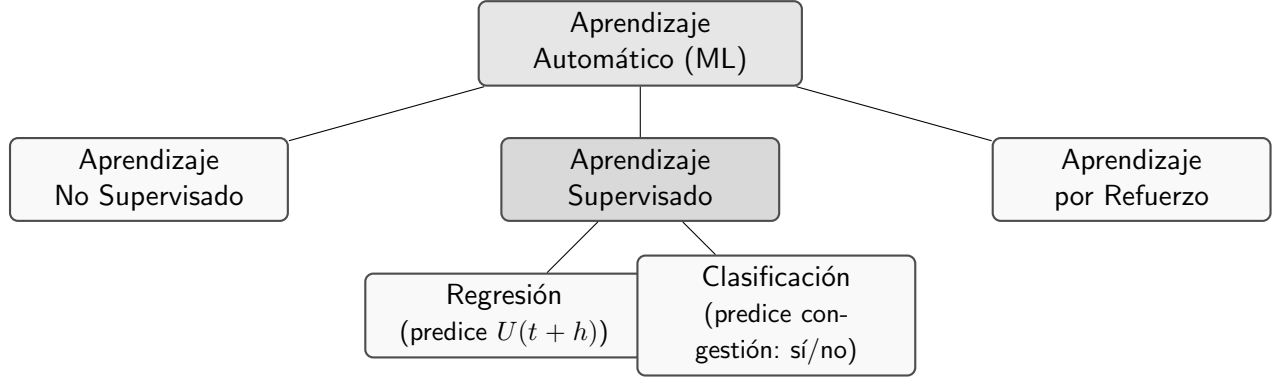


Figura 2.3: Ubicación del problema de predicción de congestión dentro de los paradigmas de Machine Learning.

2.2.2. Modelos de series temporales (ARIMA)

El modelo **ARIMA** (*AutoRegressive Integrated Moving Average*), desarrollado dentro del marco metodológico propuesto por Box y Jenkins, constituye uno de los enfoques estadísticos más consolidados para el análisis y predicción de series temporales [10]. Su nombre resume sus tres componentes constitutivos:

- **AR (AutoRegresivo)**: el valor de la serie en un instante dado se modela como una combinación lineal de sus propios valores pasados, con un orden p que indica cuántos rezagos (*lags*) se consideran.
- **I (Integrado)**: aplica un proceso de diferenciación de orden d a la serie original para eliminar tendencias y volverla estacionaria, requisito necesario para que el modelo funcione correctamente.
- **MA (Media Móvil)**: el valor de la serie se modela también en función de los errores de predicción pasados, con un orden q .

Un modelo ARIMA se denota comúnmente como $ARIMA(p, d, q)$, y se expresa formalmente como:

$$\left(1 - \sum_{i=1}^p \phi_i L^i\right) (1 - L)^d X_t = \left(1 + \sum_{i=1}^q \theta_i L^i\right) \varepsilon_t \quad (2.10)$$

donde X_t es la serie temporal de interés (en este caso, la utilización del enlace en el instante t), L es el operador de rezago, ϕ_i y θ_i son los coeficientes autorregresivos y de media

móvil respectivamente, y ε_t representa el término de error o ruido blanco. Este modelo resulta especialmente útil como **línea base** (*baseline*) dentro de esta tesis: al tratarse de un método estadístico clásico, bien establecido y computacionalmente económico, permite establecer un punto de referencia mínimo de desempeño contra el cual comparar la capacidad predictiva de los modelos de aprendizaje automático más sofisticados que se presentan a continuación.

2.2.3. Modelos de aprendizaje automático (Random Forest, SVR)

A diferencia de ARIMA, que asume relaciones lineales explícitas entre los valores pasados y futuros de la serie, los modelos de aprendizaje automático clásico son capaces de capturar relaciones no lineales entre múltiples variables de entrada simultáneamente, lo cual resulta particularmente valioso cuando se dispone de varias métricas de red (utilización, latencia, pérdida de paquetes, hora del día, día de la semana, entre otras) como posibles predictores.

Random Forest es un método de aprendizaje por conjunto (*ensemble*) que construye una multitud de árboles de decisión durante el entrenamiento, cada uno entrenado sobre una muestra aleatoria de los datos y considerando un subconjunto aleatorio de variables en cada división del árbol [11]. Para tareas de regresión, la predicción final del bosque se obtiene promediando las predicciones individuales de todos los árboles que lo componen:

$$\hat{y} = \frac{1}{T} \sum_{i=1}^T h_i(x) \quad (2.11)$$

donde T es el número total de árboles y $h_i(x)$ es la predicción del árbol i -ésimo para la entrada x . Esta estrategia de combinar múltiples predictores "débiles" en un predictor colectivo robusto reduce significativamente el riesgo de sobreajuste (*overfitting*) comparado con un único árbol de decisión, y ofrece además una ventaja adicional relevante para esta tesis: la posibilidad de calcular la **importancia relativa** de cada variable de entrada, lo cual permite identificar qué métricas de red resultan más informativas para predecir la congestión.

Por su parte, **SVR** (*Support Vector Regression*) es la adaptación al problema de regresión de las Máquinas de Vectores de Soporte (SVM), y busca encontrar una función que se ajuste a los datos permitiendo un margen de tolerancia ε , penalizando únicamente aquellas predicciones cuyo error exceda dicho margen. Su capacidad para emplear funciones de *kernel* (lineales, polinomiales o de base radial) le permite modelar relaciones no lineales complejas entre las variables de red y la utilización futura del enlace, constituyendo una alternativa adicional dentro de la familia de modelos de aprendizaje automático clásico a evaluar en esta tesis.

Es imperativo señalar que, a diferencia de los modelos autorregresivos (ARIMA) o las redes neuronales recurrentes (LSTM/GRU), los algoritmos de aprendizaje automático clásico como Random Forest y SVR carecen de una representación interna del tiempo; es decir, tratan cada observación como un evento estático e independiente. Para aplicar estos algoritmos a un problema de predicción de series temporales, es necesario transformar el conjunto de datos mediante una técnica conocida como **ventanas deslizantes** (*sliding windows*) o método de rezagos (*lags*).

Bajo este enfoque, el estado futuro de la red X_{t+h} se predice no solo a partir de las métricas en el instante t , sino construyendo un vector de características que concatena un

histórico de tamaño k . Formalmente, la entrada al modelo se transforma en una tupla de la forma $[X_t, X_{t-1}, X_{t-2}, \dots, X_{t-k}]$, convirtiendo la dependencia temporal en una dependencia espacial tabular que los algoritmos clásicos pueden procesar. La determinación empírica del tamaño óptimo de esta ventana k constituirá uno de los hiperparámetros a ajustar durante la fase experimental de este trabajo.

2.2.4. Modelos de Deep Learning (LSTM, GRU)

Las redes neuronales recurrentes (RNN) fueron diseñadas específicamente para procesar datos secuenciales, manteniendo una suerte de "memoria interna" que se actualiza en cada paso temporal. Sin embargo, las RNN convencionales sufren de un problema conocido como el desvanecimiento del gradiente (*vanishing gradient*), que les impide aprender de manera efectiva dependencias que abarcan largos periodos de tiempo —una limitación especialmente relevante para el tráfico de red, donde los patrones de congestión pueden depender de comportamientos observados varios minutos u horas atrás.

Las **LSTM** (*Long Short-Term Memory*) fueron propuestas precisamente para resolver esta limitación, mediante la introducción de una estructura interna de *compuertas* (*gates*) que regulan de manera explícita qué información se conserva, se actualiza o se descarta a lo largo del tiempo [12]. Cada unidad LSTM mantiene un estado de celda c_t que actúa como memoria de largo plazo, y tres compuertas —de olvido, de entrada y de salida— que controlan el flujo de información:

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f) \quad (\text{compuerta de olvido}) \quad (2.12)$$

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i) \quad (\text{compuerta de entrada}) \quad (2.13)$$

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o) \quad (\text{compuerta de salida}) \quad (2.14)$$

donde σ es la función de activación sigmoide, x_t es la entrada en el instante t , h_{t-1} es el estado oculto anterior, y W y b son los pesos y sesgos aprendidos durante el entrenamiento. Esta arquitectura permite a la red aprender de forma selectiva qué eventos pasados —por ejemplo, una ráfaga de sincronización de Nextcloud ocurrida varios minutos atrás— siguen siendo relevantes para predecir el estado de la red en el presente.

Las **GRU** (*Gated Recurrent Units*) constituyen una variante simplificada de las LSTM, propuesta posteriormente con el objetivo de reducir la complejidad computacional del modelo sin sacrificar significativamente su capacidad de capturar dependencias temporales de largo plazo [13]. Las GRU combinan las compuertas de olvido y entrada en una única *compuerta de actualización*, y prescinden del estado de celda independiente que caracteriza a las LSTM, resultando en un modelo con menos parámetros entrenables y, por tanto, potencialmente más eficiente de entrenar sobre conjuntos de datos de tamaño moderado como los descritos en la Sección 4.4. La comparación entre LSTM y GRU en cuanto a precisión y costo computacional se abordará empíricamente en el Capítulo 6 de esta tesis.

2.2.5. Métricas de evaluación

Para poder comparar de manera objetiva el desempeño de los distintos modelos presentados en esta sección —ARIMA, Random Forest, SVR, LSTM y GRU— es necesario definir

métricas cuantitativas de error, aplicables tanto a la fase de validación como a la comparación final de resultados en el Capítulo 6. Sea y_i el valor real observado, $\hat{y}_i$ el valor predicho por el modelo, y n el número total de observaciones evaluadas.

RMSE (*Root Mean Squared Error*, raíz del error cuadrático medio):

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2} \quad (2.15)$$

Al elevar al cuadrado cada error antes de promediar, el RMSE penaliza de manera desproporcionada los errores grandes, lo cual resulta conveniente en el contexto de esta tesis: un modelo que falle de forma drástica en predecir un evento de congestión severa es considerablemente más problemático que uno con pequeños errores distribuidos de manera uniforme.

MAE (*Mean Absolute Error*, error absoluto medio):

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i| \quad (2.16)$$

A diferencia del RMSE, el MAE pondera todos los errores de manera lineal e igualitaria, por lo que resulta menos sensible a valores atípicos (*outliers*) y ofrece una interpretación más directa: representa, en promedio, cuánto se equivoca el modelo en las mismas unidades que la variable original (en este caso, puntos porcentuales de utilización).

MAPE (*Mean Absolute Percentage Error*, error porcentual absoluto medio):

$$\text{MAPE} = \frac{100\%}{n} \sum_{i=1}^n \left| \frac{y_i - \hat{y}_i}{y_i} \right| \quad (2.17)$$

El MAPE expresa el error como un porcentaje relativo al valor real, lo que facilita su interpretación independientemente de la escala de la variable y permite comparar el desempeño de un mismo modelo entre el dataset primario y el dataset de validación externa (MAWI), a pesar de que ambos operan a escalas de tráfico sustancialmente distintas, tal como se estableció en la Sección 4.4. Como limitación, esta métrica no está definida cuando $y_i = 0$, situación que deberá considerarse durante su implementación si la utilización del enlace llegara a registrar valores nulos en algún intervalo.

Como se estableció en la Sección 2.4, el problema predictivo se abordará tanto desde una perspectiva de regresión como de clasificación. Mientras que las métricas anteriores (RMSE, MAE, MAPE) permiten cuantificar el error en la predicción del valor continuo de utilización, resulta imperativo introducir un conjunto complementario de métricas para evaluar el enfoque de clasificación binaria (congestión vs. operación normal).

Dado que la congestión de red constituye un evento anómalo —es decir, el enlace opera por debajo del umbral U_{th} la mayor parte del tiempo—, el conjunto de datos de operación se encuentra inherentemente desbalanceado. En estos escenarios, utilizar la exactitud global (*accuracy*) resulta metodológicamente engañoso, ya que un modelo que siempre prediga operación normal obtendría un puntaje artificialmente alto sin haber aprendido a detectar la congestión. En su lugar, el desempeño del clasificador se evaluará a partir de la matriz de confusión, empleando las siguientes métricas basadas en los Verdaderos Positivos (*VP*), Verdaderos Negativos (*VN*), Falsos Positivos (*FP*) y Falsos Negativos (*FN*):

Precisión (*Precision*): Cuantifica la proporción de predicciones positivas que resultaron correctas. En el contexto de esta tesis, indica qué porcentaje de las advertencias de congestión emitidas por el modelo correspondieron a eventos reales. Una alta precisión es crucial para minimizar las falsas alarmas, las cuales podrían inducir al administrador o al sistema autónomo a redirigir tráfico o escalar recursos de manera innecesaria.

$$\text{Precisión} = \frac{VP}{VP + FP} \quad (2.18)$$

Exhaustividad (*Recall* o Tasa de Verdaderos Positivos): Mide la proporción de eventos positivos reales que fueron identificados correctamente por el algoritmo. Representa la capacidad del sistema para detectar exitosamente los incidentes de congestión, evitando que estos pasen desapercibidos (falsos negativos) y terminen degradando la calidad del servicio al usuario final.

$$\text{Exhaustividad} = \frac{VP}{VP + FN} \quad (2.19)$$

F1-Score: Se define como la media armónica entre la precisión y la exhaustividad. Esta métrica es la herramienta de evaluación primaria en conjuntos de datos desbalanceados, ya que castiga severamente a los modelos que sobreoptimizan una métrica a expensas de la otra (por ejemplo, emitir alertas constantemente para maximizar la exhaustividad, arruinando la precisión). El F1-Score proporciona un único valor escalar que refleja la robustez y el equilibrio general del modelo para distinguir el estado real del enlace.

$$\text{F1-Score} = 2 \times \frac{\text{Precisión} \times \text{Exhaustividad}}{\text{Precisión} + \text{Exhaustividad}} \quad (2.20)$$

La Tabla 2.2 sintetiza los cinco modelos presentados en esta sección, contrastando su naturaleza, complejidad y tipo de relación que son capaces de capturar, como punto de partida para la justificación metodológica que se desarrollará en el Capítulo 4.

Modelo	Paradigma	Relaciones	Complejidad
ARIMA	Estadístico clásico	Lineales	Baja
Random Forest	ML clásico (ensemble)	No lineales	Media
SVR	ML clásico (kernel)	No lineales	Media
LSTM	Deep Learning (RNN)	No lineales, largo plazo	Alta
GRU	Deep Learning (RNN)	No lineales, largo plazo	Media-Alta

Cuadro 2.2: Resumen comparativo de los modelos de predicción considerados en esta tesis.

Capítulo 3

Estado del Arte

El presente capítulo tiene como propósito situar esta tesis dentro del cuerpo de conocimiento existente sobre predicción de tráfico y congestión en redes de computadoras, identificando qué se ha hecho, con qué herramientas, bajo qué condiciones y con qué resultados, para de esa manera justificar con evidencia —y no solo con intuición— las decisiones metodológicas que se tomarán en el Capítulo 4.

La revisión se organiza siguiendo la evolución cronológica y metodológica que ha caracterizado a este campo de estudio: se inicia con los trabajos fundamentados en métodos estadísticos clásicos (Sección 3.1), que dominaron la literatura durante las décadas de 1990 y 2000; se continúa con la incorporación de técnicas de Machine Learning clásico (Sección 3.2), que ganaron protagonismo conforme creció la disponibilidad de datos y de poder de cómputo; se revisan después los enfoques basados en Deep Learning (Sección 3.3), que representan el estado actual de mayor sofisticación técnica; se examinan las herramientas de monitoreo y los conjuntos de datos que la comunidad académica ha empleado para validar estos enfoques (Sección 3.4); y finalmente se sintetiza esta revisión identificando la brecha específica que esta tesis busca cubrir (Sección 3.5).

Para cada trabajo revisado se prestará atención particular a cuatro aspectos: (1) el enfoque metodológico empleado y sus fundamentos teóricos; (2) la naturaleza y origen de los datos utilizados para entrenar y validar el modelo; (3) las métricas de evaluación reportadas y los resultados obtenidos; y (4) las limitaciones reconocidas por los propios autores o identificables desde una lectura crítica, que resultan relevantes para el diseño de esta investigación. Este último punto merece una precisión importante: el objetivo de señalar limitaciones no es desacreditar el trabajo de otros investigadores, sino identificar con precisión los aspectos que aún no han sido resueltos de manera satisfactoria por la literatura existente, y que constituyen la motivación concreta para el aporte de esta tesis.

Es importante advertir, además, que la mayoría de los trabajos revisados no abordan la predicción de *congestión* como un evento discreto y accionable —tal como se definió formalmente en la Sección 2.1.2 de esta tesis (Ecuación 2.2)—, sino que se concentran en la predicción general del *volumen de tráfico* como una serie temporal continua. Esta distinción, aparentemente sutil, resulta central para posicionar la contribución de este trabajo, y se retomará con mayor detalle en la síntesis del estado del arte (Sección 3.5).

3.1. Revisión de trabajos sobre predicción de tráfico con métodos estadísticos

Los métodos estadísticos clásicos, y en particular la familia de modelos ARIMA presentada en la Sección 2.2.2, constituyeron durante largo tiempo la herramienta predominante para el análisis y predicción de tráfico de red, debido a su fundamento matemático bien establecido y a su relativa simplicidad de implementación frente a los enfoques de aprendizaje automático que se popularizarían posteriormente.

Uno de los trabajos pioneros en esta línea es el de Moayedi y Masnadi-Shirazi [14], quienes propusieron el empleo de un modelo ARIMA básico no solo con fines de predicción, sino también de detección de anomalías en el tráfico de red. Su aporte central consistió en descomponer la señal de tráfico observada en dos componentes: una parte de variación normal, que sigue un patrón predecible y puede ser capturada adecuadamente por el modelo ARIMA, y una parte anómala, correspondiente a cambios súbitos e impredecibles que el modelo no logra anticipar. Bajo esta lógica, los autores plantean que las desviaciones significativas entre el valor predicho por ARIMA y el valor real observado pueden emplearse como un indicador de eventos anómalos, entre los cuales se encontrarían episodios de saturación o congestión no habitual. Para los fines de esta tesis, este trabajo resulta relevante porque ilustra una limitación estructural de los modelos puramente estadísticos: su capacidad predictiva depende críticamente de que el tráfico se comporte de manera similar a su historia reciente, una condición que —como se discutió en la Sección 2.1.2— no necesariamente se cumple durante los episodios de congestión más severos, que suelen originarse precisamente por comportamientos atípicos del tráfico.

Una línea de trabajo posterior buscó superar esta limitación combinando ARIMA con técnicas complementarias capaces de capturar la componente no lineal del tráfico que el modelo estadístico puro no logra representar. Tian [15] propone un enfoque híbrido en el que ARIMA se emplea para modelar y predecir la componente lineal de la serie de tráfico, mientras que un filtro de Volterra adaptativo —optimizado mediante un algoritmo de enjambre de partículas— se encarga de modelar la componente no lineal residual que queda después de sustraer la predicción de ARIMA de la serie original. Los resultados reportados por el autor muestran una mejora en la precisión de predicción respecto del uso de ARIMA de manera aislada, lo cual respalda una idea central que se retomará en la síntesis de este capítulo: los modelos puramente lineales, si bien valiosos como referencia, resultan insuficientes por sí solos para capturar la complejidad real del tráfico de redes modernas.

Más recientemente, Kochetkova et al. [16] evaluaron el desempeño de modelos SARIMA (ARIMA estacional) y de suavizamiento exponencial de Holt-Winters para la predicción a corto plazo de tráfico en redes móviles de quinta generación (5G), motivados por la necesidad de una gestión dinámica y eficiente de los recursos de radio en este tipo de infraestructuras. Este trabajo es particularmente pertinente para la presente tesis por dos razones: primero, porque emplea explícitamente la componente estacional del tráfico (patrones que se repiten en ciclos horarios, diarios o semanales), un fenómeno que también se espera observar en el dataset primario de esta investigación, dada la coexistencia de tráfico residencial, de videovigilancia y de sincronización en la nube descrita en la Sección 4.4; y segundo, porque data de un periodo reciente (2023), lo cual evidencia que, a pesar del auge de los modelos de Deep Learning,

los métodos estadísticos clásicos continúan siendo objeto de investigación activa y conservan vigencia práctica, particularmente como líneas base computacionalmente económicas frente a modelos más complejos.

En conjunto, estos tres trabajos —que abarcan más de una década de investigación— permiten extraer una primera conclusión relevante para esta tesis: los modelos estadísticos como ARIMA y sus variantes ofrecen un desempeño razonable y una implementación computacionalmente accesible cuando el tráfico exhibe patrones regulares y estacionarios, pero muestran limitaciones consistentes frente a comportamientos no lineales, anómalos o súbitos —que son, precisamente, los que con mayor frecuencia preceden a un evento de congestión severa. Esta observación motiva directamente la revisión de enfoques de Machine Learning clásico que se presenta en la Sección 3.2, los cuales fueron desarrollados en buena medida como respuesta a estas limitaciones.

3.2. Predicción de tráfico/congestión con Machine Learning clásico

Frente a las limitaciones de los modelos estadísticos lineales discutidas en la sección anterior, un segundo cuerpo de literatura ha explorado el empleo de algoritmos de Machine Learning clásico —presentados en la Sección 2.2.3 de esta tesis— para abordar el problema de predicción de congestión, aprovechando su capacidad de modelar relaciones no lineales entre múltiples variables de red simultáneamente.

Un trabajo directamente pertinente para esta investigación es el de Sneha et al. [17], quienes abordaron específicamente el problema de predecir congestión a nivel de enrutador con el fin de anticipar la pérdida de paquetes antes de que esta ocurriera. Los autores entrenaron y compararon cuatro algoritmos de clasificación —K-Vecinos Más Cercanos (KNN), Random Forest, Gradient Boosting y Árboles de Decisión— sobre datos de tráfico simulado, encontrando que Random Forest y KNN ofrecieron el mejor desempeño para anticipar el tipo de pérdida de paquetes que se produciría. Este trabajo resulta especialmente relevante para la presente tesis por dos razones: primero, porque aborda la congestión como un evento a nivel de dispositivo de red (el enrutador) en lugar de como una propiedad agregada de toda la red, un enfoque compatible con la definición operativa formulada en la Ecuación 2.2; y segundo, porque confirma empíricamente la superioridad de Random Forest sobre otros algoritmos clásicos para esta tarea específica, lo cual respalda su inclusión como uno de los tres modelos a implementar en esta tesis.

En un dominio relacionado mas no idéntico, Qian et al. [18] propusieron SVR-NoC, un modelo basado en Support Vector Regression para predecir la latencia de tráfico en arquitecturas de Redes en Chip (*Network-on-Chip*, NoC) —infraestructuras de interconexión a pequeña escala empleadas dentro de procesadores multinúcleo—. Si bien este trabajo no se refiere a redes de computadoras tradicionales como las que aborda esta tesis, sino a una escala de red sustancialmente distinta, resulta útil citarlo porque demuestra la viabilidad de SVR para modelar fenómenos de saturación y tiempos de espera en sistemas basados en colas, obteniendo una reducción superior al 100 % en tiempo de cómputo respecto a métodos de simulación tradicionales sin sacrificar precisión de manera significativa. Esta evidencia, aunque proveniente de un dominio distinto, refuerza la pertinencia de incluir SVR entre los

modelos de ML clásico a evaluar en el Capítulo 5, dada su comprobada capacidad para modelar fenómenos de congestión basados en teoría de colas —el mismo fundamento teórico presentado en la Sección 2.1.2 de esta tesis.

Finalmente, el trabajo de Labonne et al. [19] resulta el más cercano en espíritu al problema abordado en esta investigación, ya que se enfoca explícitamente en predecir la utilización de ancho de banda en enlaces de red con el propósito declarado de detectar y corregir la congestión antes de que esta ocurra —el mismo objetivo central planteado en el Planteamiento del Problema de esta tesis (Sección 1.1)—. Los autores generaron su propio conjunto de datos mediante una red simulada, sometida deliberadamente a cargas de tráfico variables para provocar enlaces saturados, y aplicaron un proceso de ingeniería de características para enriquecer las variables de entrada antes de entrenar y comparar tres modelos: ARIMA, un Perceptrón Multicapa (MLP) y una red LSTM. Aunque estos dos últimos modelos corresponden propiamente a la familia de Deep Learning que se revisará en la Sección 3.3, el aporte metodológico de este trabajo para la presente tesis reside en su estrategia de generación y preprocesamiento de datos: la construcción deliberada de escenarios de saturación mediante tráfico controlado ofrece un precedente metodológico directamente aplicable a la validación del dataset primario de esta investigación, descrito en la Sección 4.4, en caso de que la red del cliente no exhiba de forma natural una cantidad suficiente de episodios de congestión durante el periodo de recolección.

La revisión de estos tres trabajos permite extraer una segunda conclusión relevante para esta tesis: los modelos de Machine Learning clásico, y en particular Random Forest y SVR, han demostrado un desempeño consistentemente superior a los modelos puramente estadísticos cuando se trata de capturar relaciones no lineales en el tráfico de red, incluso en dominios de aplicación tan distintos como enrutadores IP tradicionales, redes en chip y enlaces de banda ancha simulados. No obstante, ninguno de los trabajos revisados en esta sección explota explícitamente la naturaleza *secuencial* y dependiente del tiempo del tráfico de red —es decir, el hecho de que el valor de utilización en un instante dado depende fuertemente de una ventana de observaciones pasadas, y no únicamente de un conjunto de variables independientes—. Esta limitación motiva la revisión de los modelos de Deep Learning basados en arquitecturas recurrentes, presentados en la siguiente sección, los cuales fueron diseñados precisamente para capturar este tipo de dependencias temporales.

3.3. Predicción con Deep Learning

El auge de las arquitecturas de Deep Learning presentadas en la Sección 2.2.4 —particularmente las redes recurrentes LSTM y GRU— ha desplazado en buena medida a los enfoques de Machine Learning clásico como el foco principal de la investigación reciente en predicción de tráfico de redes, motivado por su capacidad de capturar dependencias temporales de largo alcance sin necesidad de un diseño manual de características tan extenso como el que requieren los modelos revisados en la Sección 3.2.

Uno de los primeros trabajos en establecer una comparación sistemática entre arquitecturas de redes neuronales aplicadas específicamente al tráfico de redes de computadoras —y no de otros dominios— es el de Oliveira, Barbar y Soares [20], quienes evaluaron cuatro enfoques distintos: un Perceptrón Multicapa (MLP) entrenado con retropropagación estándar, una va-

riante con retropropagación resiliente (Rprop), una red neuronal recurrente (RNN) simple, y un Autoencoder Apilado (*Stacked Autoencoder*, SAE) como representante de arquitecturas de aprendizaje profundo más complejas. De manera notable, y hasta cierto punto contraintuitiva, los autores encontraron que los modelos más simples —la RNN y el MLP— superaron en desempeño al SAE, a pesar de que este último posee una arquitectura considerablemente más profunda y compleja. Este hallazgo resulta relevante para la presente tesis porque introduce una advertencia metodológica importante: mayor complejidad arquitectónica no garantiza necesariamente mejor desempeño predictivo, especialmente cuando el volumen de datos disponible es limitado, una condición que podría presentarse en el dataset primario de esta investigación (Sección 4.4) dada la escala reducida de la red del cliente en comparación con los grandes conjuntos de datos empleados típicamente en la literatura de Deep Learning.

Precisamente sobre esta preocupación —la disponibilidad limitada de datos— profundiza el trabajo de Saha, Haque y Sidebottom [21], quienes abordan explícitamente el problema de predicción de tráfico de Internet en redes de proveedores de servicio (ISP) de tamaño reducido, donde la cantidad de datos históricos disponibles suele ser insuficiente para entrenar adecuadamente modelos de Deep Learning de gran capacidad. Los autores proponen el empleo de *transfer learning* —entrenando inicialmente modelos LSTM (en sus variantes Seq2Seq y Seq2Seq con mecanismo de atención) sobre un conjunto de datos extenso proporcionado por Juniper Networks, para posteriormente adaptarlos a conjuntos de datos considerablemente más pequeños— junto con técnicas de aumento de datos basadas en transformada wavelet. Este trabajo es particularmente pertinente para esta tesis porque su motivación —redes pequeñas con historial de datos limitado— describe con precisión la situación del dataset primario de esta investigación, recolectado durante un periodo de apenas (NUMERO FINAL) días sobre una red de escala reducida. Sus hallazgos sugieren que, de presentarse limitaciones de datos similares, estrategias como el *transfer learning* o el aumento de datos podrían explorarse como líneas de trabajo futuro si el desempeño de los modelos LSTM y GRU propuestos en el Capítulo 4 resultara insuficiente sobre el dataset primario.

Finalmente, el trabajo de Santos et al. [22] ofrece la comparación más directamente aplicable a las decisiones metodológicas de esta tesis, al confrontar explícitamente el desempeño de LSTM contra GRU —los dos modelos de Deep Learning seleccionados en la Sección 2.2.4 de este trabajo— para la predicción de tráfico de Internet móvil. Los autores emplearon dos meses de datos reales del conjunto Telecom Italia correspondientes al área metropolitana de Milán —una duración de recolección del mismo orden de magnitud que la empleada para el dataset primario de esta tesis (Sección 4.4)—, agrupando primero las series de tráfico mediante *clustering* de K-Means y optimizando posteriormente los hiperparámetros de cada modelo mediante búsqueda en malla (*Grid Search*). Sus resultados, evaluados mediante RMSE, indican que la LSTM superó a la GRU en la mayoría de los agrupamientos analizados, aunque los autores destacan que el desempeño relativo varió de manera apreciable entre distintas zonas geográficas de la ciudad, sugiriendo que la superioridad de un modelo sobre otro no es universal, sino que depende de las características particulares del tráfico analizado. Este hallazgo es directamente relevante para el diseño experimental de esta tesis: motiva la necesidad de evaluar tanto LSTM como GRU sobre *ambos* conjuntos de datos —el dataset primario recolectado de forma manual a través de un cliente y el conjunto de validación MAWI—, en lugar de asumir de antemano la superioridad de un modelo sobre el otro, dado que la literatura revisada aquí no ofrece una conclusión unívoca al respecto.

La revisión de estos tres trabajos, en conjunto con los de la Sección 3.2, permite extraer una tercera y última conclusión relevante para el diseño metodológico de esta tesis: si bien las arquitecturas de Deep Learning —y en particular las redes recurrentes como LSTM y GRU— han demostrado capacidades notables para modelar la naturaleza secuencial del tráfico de red, su desempeño superior frente a modelos más simples no está garantizado de manera universal, sino que depende críticamente de factores como el volumen de datos disponible, la naturaleza del tráfico analizado y la elección adecuada de hiperparámetros. Esta observación refuerza la pertinencia del diseño comparativo adoptado en esta tesis —evaluando ARIMA, Random Forest y LSTM/GRU de manera simultánea sobre los mismos conjuntos de datos—, en lugar de asumir a priori que el modelo más sofisticado será necesariamente el de mejor desempeño. Esta idea se retoma y se sintetiza, junto con las conclusiones de las Secciones 3.1 y 3.2, en la síntesis del estado del arte que cierra este capítulo (Sección 3.5).

3.4. Herramientas de monitoreo y datasets utilizados

Además de los modelos predictivos revisados en las secciones anteriores, resulta igualmente relevante examinar *cómo* la comunidad académica ha obtenido los datos que alimentan dichos modelos, ya que las decisiones de instrumentación —qué herramienta emplear, con qué granularidad y sobre qué tipo de red— condicionan directamente la calidad y el alcance de cualquier sistema de predicción de congestión. Esta sección organiza dicha revisión en dos partes: primero, las herramientas y protocolos de monitoreo empleados típicamente en la literatura; y segundo, una síntesis de los conjuntos de datos concretos utilizados por los trabajos revisados en las Secciones 3.1 a 3.3, incluyendo el propio MAWI Working Group Traffic Archive adoptado como dataset de validación en esta tesis (Sección 4.4).

Taxonomía de las técnicas de monitoreo

Como se adelantó en la Sección 2.1.3, las técnicas de monitoreo de red se dividen fundamentalmente en dos familias: el monitoreo **pasivo**, que observa el tráfico existente sin generar tráfico adicional, y el monitoreo **activo**, que inyecta tráfico de prueba deliberadamente para medir el comportamiento de la red. La Figura 3.1 sintetiza esta taxonomía junto con las herramientas y protocolos más representativos de cada familia, varios de los cuales fueron adoptados directamente en la metodología de recolección de datos de esta tesis (Sección 4.4).

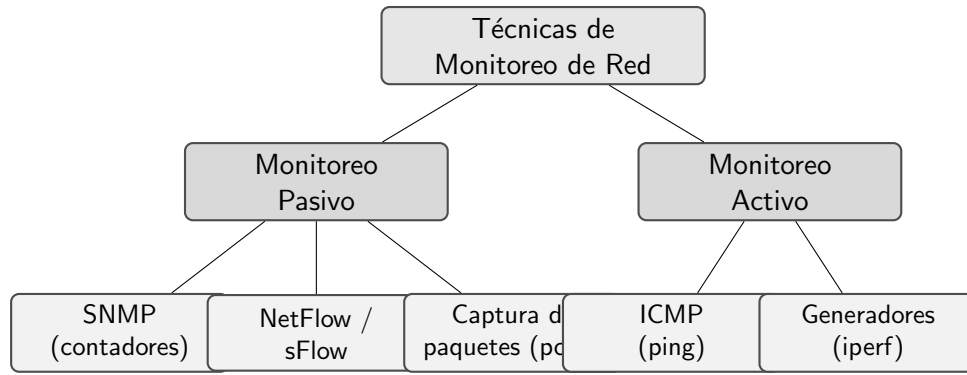


Figura 3.1: Taxonomía de las técnicas de monitoreo de red empleadas en la literatura revisada.

La Tabla 3.1 amplía esta taxonomía, contrastando cada herramienta en términos de la sobrecarga (*overhead*) que introduce en la red, la granularidad temporal que permite obtener, y el tipo de información que provee.

Herramienta	Tipo	Overhead	Información obtenida
SNMP	Pasivo	Muy bajo	Contadores agregados de utilización, errores, descartes (Sección 2.1.3)
NetFlow / sFlow	Pasivo	Medio*	Metadatos por flujo: IPs, puertos, bytes, paquetes. *El overhead real depende críticamente de la tasa de muestreo (<i>sampling rate</i>) configurada en el equipo.
Captura de paquetes (pcap)	Pasivo	Medio-Alto	Registro completo de paquetes; requiere procesamiento posterior (ej. <i>tshark</i>)
ICMP (ping)	Activo	Bajo	Latencia (RTT) y pérdida de paquetes hacia un destino específico
Generadores de tráfico (iperf)	Activo	Alto	Throughput máximo alcanzable; puede inducir congestión artificialmente

Cuadro 3.1: Comparación de herramientas de monitoreo empleadas en la literatura revisada.

Como se estableció en la Sección 4.4, esta tesis adopta un enfoque predominantemente pasivo (SNMP para utilización, complementado con mediciones activas ligeras mediante ICMP para latencia y pérdida de paquetes), evitando deliberadamente el uso de generadores de tráfico como `iperf` sobre la red del cliente, dado que estos podrían introducir una carga adicional que alterara artificialmente las condiciones reales de operación que se busca modelar.

Síntesis de datasets empleados en la literatura

La Tabla 3.2 sintetiza los conjuntos de datos empleados por los trabajos revisados en las Secciones 3.1 a 3.3 de este capítulo, permitiendo identificar patrones respecto al tipo de red, la duración de recolección y la naturaleza —real o simulada— de cada dataset.

Trabajo	Dataset	Naturaleza	Escala	Duración
Moayedi & Masnadi-Shirazi [14]	Tráfico propio	Real	No especificada	No especificada
Kochetkova et al. [16]	Red móvil 5G	Real	Operador móvil	Corto plazo
Sneha et al. [17]	Simulación	Simulado	Enrutador individual	No aplica
Labonne et al. [19]	Red simulada (VirtualBox)	Simulado	Red de laboratorio	No especificada
Santos et al. [22]	Telecom Italia (Milán)	Real	Metropolitana	2 meses
Saha et al. [21]	Juniper Networks + ISPs pequeños	Real	ISP (variable)	Variable
Esta tesis (primario)	Red del Cliente	Real	Red pequeña/PYMENA	(numero fínal) días
Esta tesis (validación)	MAWI Archive [23]	Real	Tránsito 1-10 Gbps	Diaria (continua)

Cuadro 3.2: Comparación de datasets empleados en la literatura revisada frente a los datasets adoptados en esta tesis.

Dos observaciones se desprenden de esta comparación. En primer lugar, la duración adoptada para el dataset primario de esta tesis es del mismo orden de magnitud que la empleada por Santos et al. [22], uno de los trabajos más metodológicamente cercanos revisados en este capítulo, lo cual respalda la suficiencia de dicho periodo para capturar patrones cíclicos diarios y semanales de tráfico. En segundo lugar, se observa que una proporción considerable de los trabajos revisados —particularmente aquellos enfocados en ML clásico— recurren a datos simulados en lugar de tráfico real, una decisión metodológica que facilita el control experimental pero que limita la validez de los hallazgos frente a condiciones de operación reales. Esta observación refuerza el valor diferencial del enfoque híbrido adoptado en esta tesis, que combina datos reales de dos naturalezas y escalas distintas en lugar de recurrir a tráfico simulado o a una única fuente.

3.5. Síntesis del estado del arte y brecha a cubrir

Las secciones anteriores de este capítulo revisaron nueve trabajos distintos, organizados cronológica y metodológicamente en tres grandes familias: métodos estadísticos clásicos (Sección 3.1), Machine Learning clásico (Sección 3.2) y Deep Learning (Sección 3.3), complementados con una revisión de las herramientas de instrumentación y los datasets empleados por dichos trabajos (Sección 3.4). El propósito de esta sección final es triple: primero, consolidar en un solo lugar los hallazgos dispersos a lo largo del capítulo; segundo, analizar de manera sistemática el *qué*, *por qué*, *cómo*, *para qué* y *cuándo* de la literatura revisada, con el fin de identificar patrones que no resultan evidentes al leer cada trabajo de manera aislada; y tercero —y más importante—, delimitar con precisión la brecha específica que esta tesis busca cubrir, así como el aporte concreto que este trabajo ofrece de vuelta a la comunidad académica.

Tabla maestra de trabajos revisados

La Tabla 3.3 reúne los nueve trabajos citados en este capítulo, contrastando su enfoque metodológico, la naturaleza de los datos empleados, su aporte principal, y —el elemento más relevante para los fines de esta síntesis— la limitación específica que, desde la perspectiva de esta tesis, dicho trabajo deja abierta.

Trabajo	Año	Enfoque	Aporte principal	Limitación relevante
Moayed & Masnadi-Shirazi [14]	2008	ARIMA	Detección de anomalías vía residuos de predicción	No captura no linealidades; congestión inferida indirectamente
Tian [15]	2021	ARIMA + Volterra	Mejora de ARIMA con componente no lineal	Sigue centrado en volumen de tráfico, no en congestión como evento
Kochetkova et al. [16]	2023	SARIMA / Holt-Winters	Explota estacionalidad en redes 5G	Sin comparación contra ML/DL en el mismo estudio
Sneha et al. [17]	2020	Random Forest, KNN, GB, DT	Predicción de congestión a nivel de enrutador	Datos simulados; sin validación en tráfico real
Qian et al. [18]	2016	SVR	Modelado de latencia basado en colas	Dominio distinto (NoC, no redes IP)
Labonne et al. [19]	2021	ARIMA, MLP, LSTM	Framework completo de generación y predicción de utilización	Red simulada; sin validación externa
Oliveira et al. [20]	2016	MLP, RNN, SAE	Comparación de arquitecturas de red neuronal	Mayor complejidad (SAE) no garantiza mejor desempeño
Saha et al. [21]	2024	LSTM + Transfer Learning	Aborda datasets pequeños en ISPs	No compara contra modelos estadísticos ni ML clásico
Santos et al. [22]	2022	LSTM vs. GRU	Comparación directa LSTM/GRU en tráfico real (2 meses)	Un solo paradigma (DL); sin ARIMA ni ML clásico

Cuadro 3.3: Síntesis comparativa de los trabajos revisados en el Estado del Arte.

Evolución cronológica y metodológica

La Figura 3.2 presenta una línea de tiempo que sitúa los trabajos revisados —junto con los hitos teóricos fundamentales presentados en el Marco Teórico de esta tesis (Capítulo 2)— dentro de la evolución histórica del campo, permitiendo visualizar de manera inmediata la transición gradual de los métodos estadísticos hacia el Machine Learning clásico y, finalmente, hacia el Deep Learning.

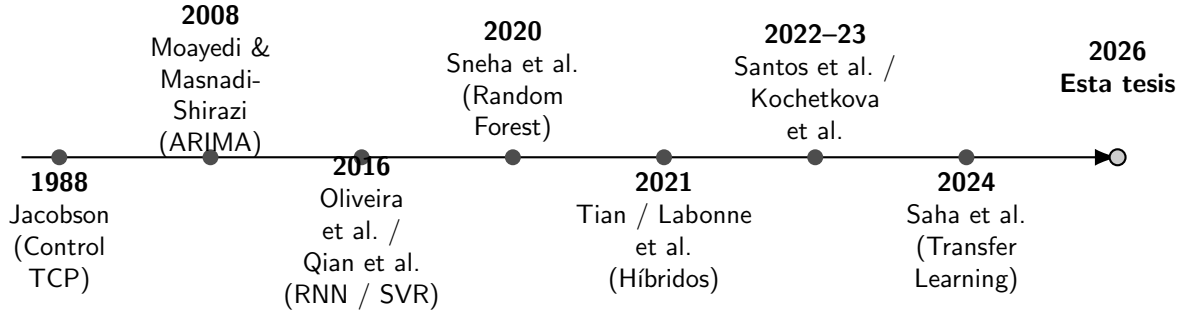


Figura 3.2: Línea de tiempo de la evolución metodológica en predicción de tráfico/congestión de red.

Esta visualización revela un patrón claro: durante casi dos décadas (1988–2008) predominaron los métodos estadísticos; a partir de 2016 comienza una transición gradual y simultánea hacia el ML clásico y las primeras arquitecturas de Deep Learning; y a partir de 2020 los tres paradigmas coexisten activamente en la literatura, sin que ninguno haya desplazado por completo a los anteriores. Esta coexistencia —más que una sustitución progresiva— es, como se argumentará más adelante, uno de los elementos que justifica el diseño comparativo de esta tesis.

Análisis por dimensiones

¿Qué se ha investigado? La literatura revisada se ha concentrado abrumadoramente en la predicción del **volumen de tráfico** como una serie temporal continua (Moayedi & Masnadi-Shirazi [14]; Tian [15]; Kochetkova et al. [16]; Oliveira et al. [20]; Santos et al. [22]), tratando la congestión como una consecuencia implícita de dicho volumen más que como un evento a predecir de manera directa y explícita. Solo dos de los nueve trabajos revisados —Sneha et al. [17] y, de manera indirecta, Qian et al. [18]— abordan la congestión como una variable de interés discreta y directamente accionable, más cercana al enfoque de clasificación binaria establecido en la definición operativa de esta tesis (Ecuación 2.2).

¿Por qué se ha investigado de esta manera? La motivación predominante en la literatura revisada es la optimización de la gestión de recursos de red —asignación dinámica de ancho de banda, planeación de capacidad, mejora de la Calidad de Servicio (QoS)— más que la prevención proactiva de eventos específicos de degradación del servicio. Esta orientación explica por qué la mayoría de los trabajos evalúan sus modelos mediante métricas de error continuo (RMSE, MAE, MAPE) en lugar de métricas propias de un problema de clasificación (precisión, exhaustividad, F1-score), a pesar de que estas últimas resultarían más informativas para un caso de uso de alertamiento temprano como el que plantea esta tesis.

¿Cómo se ha investigado? Metodológicamente, se observa una fragmentación notable: cada trabajo revisado evalúa típicamente **un solo paradigma** de modelado —o, en el mejor de los casos, variantes dentro de un mismo paradigma, como Santos et al. [22] comparando LSTM contra GRU, ambos de Deep Learning—. Ningún trabajo de los revisados en este

capítulo compara de manera simultánea, sobre el mismo conjunto de datos y bajo las mismas condiciones experimentales, representantes de los tres paradigmas (estadístico, ML clásico y Deep Learning), lo cual dificulta extraer conclusiones comparativas robustas a partir de la literatura existente de forma aislada. Asimismo, como se documentó en la Tabla 3.2 de la Sección 3.4, una proporción considerable de los trabajos de ML clásico recurre a datos simulados (Sneha et al. [17]; Labonne et al. [19]), mientras que los trabajos con datos reales rara vez validan sus hallazgos en más de un escenario de red.

¿Para qué se ha investigado? La finalidad última declarada por la mayoría de los trabajos revisados es habilitar una gestión de red proactiva en lugar de reactiva, alineada con la motivación general de esta tesis expuesta en el Planteamiento del Problema (Sección 1.1). Sin embargo, pocos trabajos —con la notable excepción de Sneha et al. [17]— formulan explícitamente un umbral operativo o un mecanismo de alertamiento accionable; la mayoría se detiene en la demostración de la capacidad predictiva del modelo, la mayoría se detiene en la demostración de la capacidad predictiva del modelo, sin traducir dicha capacidad en un criterio de decisión concreto. En contraste, contar con un umbral predictivo accionable como U_{th} permite a un administrador o sistema automatizado tomar medidas preventivas reales en la infraestructura, tales como la inyección dinámica de reglas de *firewall* para limitar tráfico no esencial, el ajuste de métricas en protocolos de enrutamiento para desviar flujos, o la re-dirección de tráfico crítico a través de túneles seguros alternativos antes de que la saturación provoque la caída de servicios esenciales como el monitoreo remoto.

¿Cuándo se ha investigado y hacia dónde se dirige el campo? Como se ilustra en la Figura 3.2, la investigación más reciente (2022–2024) muestra dos tendencias emergentes particularmente relevantes para esta tesis: por un lado, una preocupación creciente por la aplicabilidad práctica de estos modelos en escenarios de datos limitados (Saha et al. [21]), reconociendo que gran parte de la literatura previa asume la disponibilidad de conjuntos de datos extensos poco representativos de redes pequeñas o medianas; y por otro, comparaciones cada vez más rigurosas entre arquitecturas específicas de Deep Learning (Santos et al. [22]) que evidencian que la superioridad de un modelo sobre otro no es universal, sino dependiente del contexto de red particular.

A partir del análisis anterior es posible formular con precisión la brecha que esta tesis busca cubrir, articulada en tres componentes específicos:

1. **Brecha conceptual:** la literatura revisada predice predominantemente el *volumen* de tráfico como serie continua, mientras que esta tesis predice la *congestión* como evento discreto y accionable, mediante la definición operativa formalizada en la Ecuación 2.2 (Sección 2.1.2), permitiendo abordar el problema tanto desde la regresión como desde la clasificación (Sección 2.2.1).
2. **Brecha metodológica:** ningún trabajo revisado compara de manera simultánea y bajo condiciones experimentales controladas representantes de los tres paradigmas de modelado —estadístico (ARIMA), ML clásico (Random Forest) y Deep Learning (LSTM/GRU)—, tal como se plantea en el diseño metodológico de esta tesis (Capítulo 4).

3. **Brecha de validación:** la mayoría de los trabajos revisados se apoyan en un único dataset, ya sea real o simulado, sin contrastar sus hallazgos frente a escenarios de red de naturaleza o escala distinta. Esta tesis, mediante el enfoque híbrido descrito en la Sección 4.4 —combinando la red real de escala reducida de un cliente con el archivo de tráfico de gran escala MAWI—, aborda directamente esta limitación, en línea con la recomendación implícita de trabajos como el de Saha et al. [21] respecto a la necesidad de validar modelos más allá de un único escenario de datos.

La Tabla 3.4 sintetiza visualmente este posicionamiento, contrastando las características metodológicas predominantes en la literatura revisada frente al diseño adoptado en esta tesis.

Dimensión	Literatura revisada (predominante)	Esta tesis
Variable objetivo	Volumen de tráfico (continuo)	Congestión (discreta, vía U_{th})
Paradigmas comparados	Uno solo, o variantes de uno solo	Tres (estadístico, ML clásico, DL) simultáneamente
Naturaleza de los datos	Real o simulado, una sola fuente	Real, dos fuentes complementarias (híbrido)
Escala de validación	Una sola escala de red	Dos escalas contrastantes (PYME y tránsito de gran escala)
Umbral accionable	Raramente explícito	Definido formalmente (U_{th} , Ecuación 2.2)

Cuadro 3.4: Posicionamiento metodológico de esta tesis frente a la literatura revisada.

Aporte de esta tesis a la literatura existente

En correspondencia con las tres brechas identificadas, esta tesis ofrece tres aportes concretos a la comunidad académica. Primero, aporta evidencia empírica comparativa entre los tres paradigmas de modelado revisados en este capítulo, evaluados bajo condiciones experimentales controladas y sobre los mismos conjuntos de datos —un tipo de comparación ausente en la literatura revisada, donde cada paradigma suele evaluarse de manera aislada—. Segundo, aporta un caso de estudio de recolección de datos reales en una red heterogénea de pequeña escala (residencial/PYME con videovigilancia, servicios en la nube y tráfico de ISP coexistiendo en el mismo enlace), un escenario significativamente menos representado en la literatura que las redes troncales de gran escala o los entornos simulados de laboratorio. Tercero, aporta una definición operativa explícita y formalizada de congestión (Ecuación 2.2), que traduce directamente la salida de los modelos predictivos en un criterio accionable para la toma de decisiones, respondiendo a la limitación de umbral accionable raramente explícito identificada en la Tabla 3.4.

Recíprocamente, esta tesis también reconoce su deuda con la literatura revisada: la definición del umbral operativo U_{th} se apoya en la evidencia de estacionalidad documentada

por Kochetkova et al. [16]; el diseño de la comparación LSTM/GRU replica la metodología validada por Santos et al. [22] sobre un periodo de recolección equivalente; y la estrategia de mitigación ante datasets de tamaño reducido, en caso de ser necesaria, se apoyaría directamente en el precedente de *transfer learning* establecido por Saha et al. [21]. De esta manera, el capítulo de Metodología que sigue a continuación (Capítulo 4) no parte de cero, sino que se construye deliberadamente sobre las fortalezas identificadas y en respuesta directa a las limitaciones documentadas a lo largo de esta revisión del estado del arte.

Capítulo 4

Metodología

Los capítulos anteriores construyeron el fundamento conceptual de esta tesis: se definió formalmente qué se entiende por congestión de red (Capítulo 2, Ecuación 2.2), y se identificaron, mediante la revisión sistemática de la literatura, tres brechas específicas que este trabajo busca cubrir (Capítulo 3, Sección 3.5). Corresponde ahora al presente capítulo dar el paso de la revisión a la ejecución: traducir esas definiciones y esa brecha identificada en un plan de trabajo preciso, replicable y justificado en cada una de sus decisiones.

A diferencia de los capítulos precedentes, en los que el objeto de estudio era la producción académica de terceros, este capítulo describe exclusivamente decisiones propias del autor: qué tipo de investigación se está realizando, cómo se define y mide operativamente cada variable, de dónde provienen los datos, cómo se procesan, qué modelos se implementan y con qué criterios se evalúan. Cada una de estas decisiones no es arbitraria, sino que responde directamente a lo establecido en capítulos anteriores —la definición operativa de congestión, el dataset híbrido descrito en la Sección 4.4, y los tres paradigmas de modelado comparados en la Sección 2.2— por lo que el lector encontrará a lo largo de este capítulo referencias constantes hacia atrás, cerrando el círculo argumentativo abierto desde el planteamiento del problema.

Es importante señalar que las decisiones metodológicas aquí documentadas no necesariamente se tomaron en un único momento ni de manera lineal; muchas de ellas —como la elección del dataset híbrido o la definición del umbral de congestión— fueron resultado de un proceso iterativo de análisis y ajuste. Sin embargo, este capítulo las presenta de manera ordenada y sistemática, tal como corresponde a un documento metodológico formal, comenzando por la caracterización general del tipo de investigación que enmarca a esta tesis.

4.1. Tipo de investigación

La presente investigación se clasifica, de acuerdo con los criterios propuestos por Hernández-Sampieri [24], en cuatro dimensiones complementarias: su finalidad, su enfoque, su diseño y su alcance. Esta caracterización no es un mero formalismo, sino que determina directamente las decisiones metodológicas que se detallan en el resto de este capítulo, desde la forma en que se recolectan los datos hasta los criterios empleados para evaluar los modelos desarrollados.

Según su finalidad: investigación aplicada

Esta tesis constituye una investigación de tipo **aplicada** (también denominada tecnológica), en tanto que su propósito no es generar conocimiento teórico nuevo sobre las técnicas de Machine Learning empleadas —ARIMA, Random Forest y LSTM/GRU, cuyos fundamentos matemáticos ya se encuentran plenamente establecidos y fueron revisados en la Sección 2.2—, sino aplicar dicho conocimiento existente a la resolución de un problema práctico y delimitado: la predicción de congestión en un escenario real de red, tal como se planteó en la Sección 1.1. El criterio de éxito de esta investigación, por tanto, no reside en la novedad matemática de los modelos empleados, sino en la evidencia empírica que se obtenga sobre su viabilidad y desempeño comparativo al resolver el problema planteado.

Según su enfoque: investigación cuantitativa

El enfoque adoptado es **cuantitativo**, ya que la totalidad de las variables involucradas —utilización del enlace, latencia, pérdida de paquetes (Sección 2.1.3)— se expresan como magnitudes numéricas susceptibles de medición directa. El análisis de los resultados se apoya estrictamente en las métricas estadísticas de error (RMSE, MAE, MAPE) y en las métricas de clasificación (Precisión, Exhaustividad, F1-Score) definidas y fundamentadas teóricamente en el Capítulo 2. No se contempla, en ningún punto del desarrollo de esta tesis, la recolección o el análisis de datos de naturaleza cualitativa.

Según su diseño: mixto secuencial (observacional y experimental)

Desde la perspectiva de su diseño, esta investigación adopta un enfoque **mixto secuencial**, compuesto por dos fases distintas que reflejan la dualidad del problema abordado:

En su primera fase (Recolección de Datos), el diseño es predominantemente **no experimental y longitudinal**. Se observa y registra el comportamiento de la red a lo largo de un periodo extendido de NUMEROFINAL días para capturar la evolución y estacionalidad del tráfico. Si bien se emplea instrumentación híbrida —incluyendo sondeos activos ligeros (ICMP) como se documentará en la Sección 4.X—, no se manipulan artificialmente las cargas de trabajo de los usuarios ni se induce deliberadamente la congestión en la infraestructura del cliente corporativo.

En su segunda fase (Modelado Predictivo), el diseño transita hacia un enfoque **experimental computacional**. El desarrollo, entrenamiento y evaluación de los modelos de Machine Learning (ARIMA, Random Forest, LSTM/GRU) requiere la manipulación sistemática y controlada de variables independientes —tales como hiperparámetros, arquitecturas de red y ventanas de rezago (*lags*)— para medir su efecto sobre las variables dependientes, representadas por las métricas de error predictivo. Esta fase se ejecuta en un entorno de trabajo aislado, garantizando que la experimentación algorítmica no impacte la red de producción.

Según su alcance: comparativo y explicativo

Finalmente, esta investigación posee un alcance **comparativo**, en tanto que su objetivo central no se limita a describir el comportamiento del tráfico de red, sino a contrastar

sistemáticamente el desempeño de tres paradigmas de modelado predictivo —estadístico, Machine Learning clásico y Deep Learning— evaluados bajo las mismas condiciones experimentales, tal como se estableció en la brecha metodológica identificada en la Sección 3.5. Este alcance comparativo se complementa con un componente **explicativo**, en la medida en que no basta con determinar *cuál* modelo ofrece el mejor desempeño, sino que se busca argumentar *por qué* dicho modelo se desempeña mejor —por ejemplo, si su ventaja se debe a la naturaleza no lineal del tráfico, a la presencia de dependencias temporales de largo plazo, o a la escala del conjunto de datos empleado—, apoyándose para ello en los fundamentos teóricos desarrollados en el Capítulo 2 y en la evidencia comparativa documentada en el Capítulo 3.

La Tabla 4.1 sintetiza esta clasificación de manera esquemática.

Dimensión	Clasificación	Justificación breve
Finalidad	Aplicada / tecnológica	Aplica técnicas de ML existentes a un problema práctico de redes
Enfoque	Cuantitativa	Variables numéricas; análisis estadístico de error
Diseño	No experimental, longitudinal	Observación pasiva de la red durante NUMEROFINAL días, sin manipulación
Alcance	Comparativo-explicativo	Contrasta tres paradigmas de modelado y explica sus diferencias de desempeño

Cuadro 4.1: Clasificación del tipo de investigación de esta tesis.

4.2. Definición operativa de congestión

De acuerdo con la Ecuación 2.2 establecida en el Marco Teórico, la operacionalización de la variable objetivo exige definir valores numéricos precisos para el umbral crítico de utilización (U_{th}) y la ventana temporal sostenida (Δt). Esta sección establece los parámetros base que gobernarán el etiquetado (*labeling*) del conjunto de datos antes de su ingesta en los modelos de Machine Learning.

Justificación del umbral U_{th}

Se adopta como valor inicial $U_{th} = 0,80$ (80 % de utilización), por dos razones convergentes. En primer lugar, desde la perspectiva teórica desarrollada en la Sección 2.1.2, la teoría de colas aplicada a redes de datos establece que el tiempo de espera promedio en un sistema de colas crece de manera proporcional a $\frac{1}{1-\rho}$, donde ρ representa la utilización del recurso [6]; esta expresión matemática diverge conforme $\rho \rightarrow 1$, y su crecimiento deja de ser marginal precisamente en la vecindad de $\rho = 0,8$, punto a partir del cual pequeños incrementos adicionales en la utilización producen incrementos desproporcionadamente grandes en el tiempo de encolado. En segundo lugar, desde la perspectiva de la práctica de la industria, un umbral de utilización entre el 70 % y el 80 % constituye una referencia ampliamente adoptada en

la operación de redes empresariales para la activación de mecanismos de gestión de colas y alertamiento temprano, un criterio que se refleja, por ejemplo, en los valores por defecto de mecanismos de descarte temprano como WRED (*Weighted Random Early Detection*) en equipos de la industria, los cuales suelen configurarse con umbrales iniciales de descarte alrededor del 80 % de ocupación de buffer.

Este valor de $U_{th} = 0,80$ funge como parámetro de control inicial (línea base) derivado de la literatura. Sin embargo, para garantizar que el modelo predictivo se ajuste a la realidad topológica de la red bajo estudio, este umbral será sometido a validación empírica durante el Análisis Exploratorio de Datos. El valor definitivo para el etiquetado del algoritmo de clasificación se fijará matemáticamente identificando el percentil exacto donde la distribución de utilización histórica comience a correlacionarse con fluctuaciones anómalas en la red.

Justificación de la ventana temporal Δt

En congruencia con la granularidad de muestreo establecida para la recolección de datos (Sección 4.4), se define $\Delta t = 5$ minutos como el intervalo mínimo sostenido durante el cual la condición $U(t) \geq U_{th}$ debe cumplirse para que un periodo se etiquete como congestión. Esta ventana, equivalente a cinco muestras consecutivas bajo una granularidad de muestreo de un minuto, responde a un balance deliberado entre dos riesgos opuestos: una ventana demasiado corta (por ejemplo, una única muestra) resulta susceptible a ráfagas instantáneas y transitorias de tráfico que no representan una congestión sostenida ni operacionalmente relevante —generando así falsos positivos que degradarían la utilidad práctica del sistema de predicción—; mientras que una ventana demasiado larga podría pasar por alto episodios de saturación breves pero significativos para aplicaciones sensibles a la latencia, como las cámaras de videovigilancia o las videollamadas mencionadas en la Sección 4.4. Al igual que con U_{th} , este valor podrá ser ajustado tras el análisis exploratorio del dataset primario si la evidencia empírica así lo sugiere.

Formulación instanciada para la etapa de preprocesamiento

Tomando como base los parámetros iniciales, la lógica de etiquetado binario que se programará en el pipeline de preprocesamiento de datos se define como:

$$\text{Congestión} \iff U(t) \geq 0,80 \quad \text{durante al menos 5 minutos consecutivos} \quad (4.1)$$

4.3. Descripción de las métricas a utilizar

Habiendo definido en la Sección 2.1.3 el fundamento teórico de las métricas de rendimiento de red, y en la Sección 4.2 el criterio operativo de congestión, corresponde ahora precisar qué subconjunto específico de esas métricas se empleará como variables de entrada (*features*) de los modelos predictivos, cuál constituye la variable objetivo, y qué variables adicionales derivadas se incorporarán para enriquecer la capacidad predictiva del sistema.

Variable objetivo

La variable objetivo (*target*) de esta tesis es, en todos los casos, el factor de utilización futuro del enlace, $U(t + h)$, donde h representa el horizonte de predicción. Dicha variable se emplea de dos formas distintas según el paradigma de modelado, conforme a lo establecido en la Sección 2.2.1:

- Como variable **continua** (tarea de regresión), para los modelos ARIMA, Random Forest/SVR y LSTM/GRU en su formulación de regresión.
- Como variable **binaria derivada** (tarea de clasificación), aplicando la Ecuación 4.1 sobre $U(t + h)$, para la formulación de clasificación de los mismos modelos.

Variables de entrada (features)

La Tabla 4.2 enumera las variables de entrada consideradas para el entrenamiento de los modelos, clasificadas en tres grupos: métricas directas (obtenidas mediante los mecanismos de instrumentación descritos en la Sección 4.4), variables temporales derivadas (que capturan patrones cíclicos sin necesidad de instrumentación adicional), y variables de rezago/ventana (que explicitan la dependencia temporal de la serie, necesaria para modelos que no procesan secuencias de manera nativa, como Random Forest y SVR).

Grupo	Variable	Tipo	Unidad / Rango	Fuente
Métricas directas	Utilización $U(t)$	Continua	Adimensional [0,1]	SNMP
	Latencia (RTT)	Continua	ms	ICMP
	Pérdida de paquetes	Continua	%	SNMP / ICMP
Temporales derivadas	Hora del día	Categórica cíclica	0–23 (codificada seno/coseno)	Timestamp
	Día de la semana	Categórica cíclica	0–6 (codificada seno/coseno)	Timestamp
	Indicador fin de semana	Binaria	{0, 1}	Timestamp
	Indicador horario laboral	Binaria	{0, 1}	Timestamp
Rezago / ventana	$U(t - 1), U(t - 2), \dots, U(t - k)$	Continua	Adimensional [0,1]	Derivada de $U(t)$
	Media móvil de U (ventana de 15 min)	Continua	Adimensional [0,1]	Derivada de $U(t)$

Cuadro 4.2: Variables de entrada consideradas para los modelos predictivos.

Conviene justificar dos decisiones reflejadas en la Tabla 4.2. Primero, las variables temporales cíclicas (hora del día, día de la semana) se codifican mediante transformaciones

seno/coseno, en lugar de emplearse directamente como enteros, con el fin de preservar su naturaleza cíclica —evitando, por ejemplo, que el modelo interprete que la hora 23 y la hora 0 se encuentran lejanas entre sí, cuando en realidad son consecutivas—:

$$x_{\text{seno}} = \sin\left(\frac{2\pi \cdot x}{x_{\text{máx}}}\right), \quad x_{\text{coseno}} = \cos\left(\frac{2\pi \cdot x}{x_{\text{máx}}}\right) \quad (4.2)$$

Segundo, las variables de rezago ($U(t-1)$ hasta $U(t-k)$) resultan indispensables para los modelos que no poseen una noción arquitectónica intrínseca de secuencia temporal —Random Forest y SVR—, ya que estos modelos tratan cada observación como independiente y requieren que la dependencia temporal se les proporcione de manera explícita como parte del vector de entrada. En contraste, las arquitecturas LSTM y GRU procesan la secuencia de observaciones de manera nativa, por lo que estas variables de rezago, si bien pueden incluirse, no resultan estrictamente necesarias para dichos modelos.

Disponibilidad de métricas por fuente de datos

Dado que esta tesis emplea dos fuentes de datos de naturaleza distinta (Sección 4.4), no todas las métricas de la Tabla 4.2 se encuentran disponibles de la misma forma en ambos casos. La Tabla 4.3 contrasta la disponibilidad y el método de obtención de cada métrica en el dataset primario frente al dataset de validación (MAWI).

Métrica	Dataset primario	Dataset de validación (MAWI)
Utilización	Directa, vía SNMP en tiempo real	Derivada de trazas .pcap agregando bytes por ventana de tiempo
Latencia (RTT)	Directa, vía sondas ICMP activas	No disponible directamente; se aproxima mediante el tiempo entre paquetes de un mismo flujo TCP
Pérdida de paquetes	Directa, vía contadores SNMP de descarte	Estimada indirectamente mediante retransmisiones TCP detectadas en la traza
Variables temporales	Directas, timestamp del muestreo	Directas, timestamp de captura de cada traza diaria

Cuadro 4.3: Disponibilidad y método de obtención de métricas según la fuente de datos.

Esta asimetría no invalida el enfoque híbrido adoptado, pero sí implica que, para el dataset de validación, algunas variables de entrada requieren un paso adicional de derivación durante el preprocesamiento, el cual se detalla en la Sección 4.5. Es fundamental advertir que esta diferencia en los métodos de recolección —particularmente entre la latencia limpia

obtenida mediante sondeos ICMP en la red privada y la latencia ruidosa inferida a través de flujos TCP en el archivo MAWI— introduce un desafío estadístico conocido como cambio de dominio (*domain shift*). Si los algoritmos de Machine Learning asimilan directamente las magnitudes crudas de estas métricas, el desempeño predictivo colapsará durante la fase de validación debido al desajuste de escalas y distribuciones. Para mitigar esta divergencia estructural, se aplicará un proceso de escalado y normalización independiente (por ejemplo, mediante *StandardScaler* o *MinMaxScaler*) a cada conjunto de datos de manera aislada. Esta transformación, que se formaliza en la etapa de preprocesamiento (Sección 4.5), garantiza que los modelos aprendan la varianza relativa y los patrones de degradación de las métricas, independientemente de la magnitud absoluta que dicten las herramientas de monitoreo subyacentes.

Pipeline de construcción de variables

La Figura 4.1 ilustra de manera esquemática el flujo mediante el cual las métricas crudas, obtenidas directamente de la instrumentación descrita en la Sección 4.4, se transforman en el vector final de variables de entrada empleado por cada modelo.

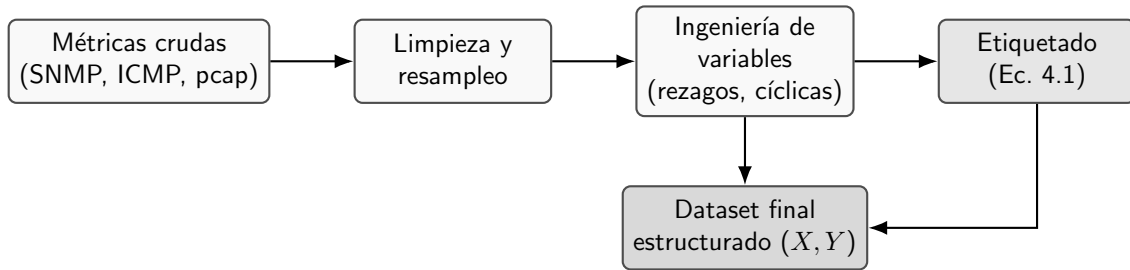


Figura 4.1: Pipeline de transformación de métricas crudas en variables de entrada y variable objetivo.

Este pipeline se aplica de manera independiente sobre cada una de las dos fuentes de datos, garantizando que el vector final de variables tenga una estructura equivalente entre ambas, condición necesaria para poder evaluar un mismo modelo entrenado sobre el dataset primario contra el conjunto de validación MAWI, tal como se planteó en el diseño comparativo de esta tesis (Sección 3.5). El detalle específico de cada etapa de este pipeline —en particular, la limpieza de datos faltantes y el resamplero de las trazas MAWI— se desarrolla con mayor profundidad en la Sección 4.5 (Preprocesamiento de datos).

4.4. Fuente de datos

Para el desarrollo de esta tesis se adoptó una estrategia híbrida de recolección de datos, combinando una fuente primaria de tráfico real capturado de forma pasiva en una red administrada directamente por el autor, con una fuente secundaria de validación externa proveniente de un archivo académico de tráfico de Internet ampliamente reconocido. Esta combinación busca equilibrar dos necesidades que, de manera aislada, ningún dataset por sí solo podría satisfacer plenamente: por un lado, la posibilidad de definir con precisión el

proceso de recolección, las métricas capturadas y el contexto operativo del enlace bajo estudio; y por otro, la necesidad de validar que los modelos desarrollados no se sobreajustan a las particularidades de un único escenario, sino que son capaces de generalizar su capacidad predictiva a condiciones de tráfico distintas en naturaleza y escala.

Dataset primario: Red Privada PYME

La fuente primaria de datos proviene de una red privada PYME sobre la cual el autor tiene acceso administrativo completo, en el contexto de un proyecto de despliegue de infraestructura. Dicha red opera bajo un enrutador MikroTik que concentra y gestiona simultáneamente múltiples servicios que compiten por el mismo enlace de salida a Internet. Es fundamental delimitar la topología física de este acceso: el tráfico hacia el exterior se enruta a través de una interfaz, lo cual condiciona intrínsecamente el perfil base de latencia, *jitter* y variabilidad estocástica del canal. Sobre esta infraestructura coexisten un sistema de videovigilancia con transmisión continua (NVR), tráfico de uso corporativo/residencial cotidiano, y un servicio de almacenamiento en la nube (Nextcloud) accesible de forma remota mediante una VPN SSTP.

Esta arquitectura de servicios genera un escenario de estrés operativo altamente propicio para la investigación de congestión. En particular, la encapsulación de tráfico TCP (como el generado por Nextcloud) dentro de un túnel VPN que también opera sobre TCP (SSTP) introduce una susceptibilidad arquitectónica al fenómeno conocido como *TCP Meltdown*. Bajo esta condición, si el enlace físico experimenta una ligera pérdida de paquetes por saturación, tanto la capa de aplicación como la capa del túnel intentan retransmitir la información de manera simultánea y descoordinada. Esta redundancia de retransmisiones amplifica la congestión original, colapsando rápidamente la capacidad del enlace mediante un patrón no lineal. La captura de estas dinámicas complejas convierte a este conjunto de datos en un entorno de validación excepcional para evaluar si los modelos de Machine Learning logran anticipar el colapso con mayor precisión que los modelos estadísticos tradicionales. Es importante señalar que, dado que los datos provienen de la infraestructura de un tercero, se obtuvo la autorización explícita del propietario de la red para realizar el monitoreo pasivo de metadatos de tráfico con fines exclusivamente académicos, en línea con buenas prácticas de investigación en redes de computadoras que involucran datos de operación reales.

Dataset secundario: MAWI Working Group Traffic Archive

Como fuente de validación externa se empleará el **MAWI Working Group Traffic Archive**, un repositorio de trazas de tráfico mantenido de forma continua desde 2006 por el proyecto WIDE, que captura de manera diaria el tráfico del enlace de tránsito entre la red académica WIDE y su proveedor ascendente en Tokio, Japón [23]. A diferencia del dataset primario, que corresponde a una red de escala reducida con un puñado de servicios concurrentes, las trazas de MAWI capturan tráfico agregado de un enlace de tránsito de gran escala (entre 1 y 10 Gbps), compuesto por miles de flujos simultáneos provenientes de múltiples organizaciones y aplicaciones. Las direcciones IP contenidas en estas trazas se encuentran anonimizadas mediante un esquema que preserva la estructura de prefijos de red (*prefix-preserving anonymization*, mediante el algoritmo Crypto-PAn), lo cual elimina cual-

quier posibilidad de identificar usuarios individuales sin comprometer la utilidad analítica de los datos para el estudio de patrones agregados de tráfico.

La incorporación de este segundo dataset responde a una limitación reconocida en la literatura sobre predicción de tráfico de red: los modelos entrenados y evaluados sobre una única fuente de datos tienden a mostrar un desempeño optimista que no necesariamente se sostiene cuando se exponen a condiciones de tráfico distintas a las observadas durante el entrenamiento. Al validar los modelos desarrollados en esta tesis sobre las trazas de MAWI —de una naturaleza y escala sustancialmente distinta a la del dataset primario—, es posible obtener evidencia más robusta sobre la capacidad de generalización de los enfoques de Machine Learning propuestos, más allá de las particularidades de un único escenario de red.

Dado que las trazas de MAWI se distribuyen en formato de captura de paquetes (`.pcap`) y no como series temporales de utilización previamente agregadas, fue necesario un procesamiento adicional para derivar de ellas las mismas métricas empleadas en el dataset primario. Este procesamiento se describe con mayor detalle en la Sección 4.5 (Preprocesamiento de datos).

Consideraciones éticas y de privacidad

En ambos casos, la recolección y el procesamiento de los datos se rigen por un principio de minimización: se capturan y almacenan únicamente los metadatos estrictamente necesarios para el análisis de utilización, latencia y pérdida de paquetes, sin acceder en ningún momento al contenido de las comunicaciones transportadas. En el caso del dataset primario, se cuenta con la autorización expresa del propietario de la infraestructura; en el caso de MAWI, el uso de los datos se limita a fines de investigación, en cumplimiento con los términos de uso establecidos por el propio archivo, los cuales explícitamente restringen su empleo a propósitos académicos y prohíben cualquier acción que comprometa la privacidad de los usuarios cuyo tráfico fue originalmente capturado.

4.5. Preprocesamiento de datos

El pipeline esquematizado en la Figura 4.1 (Sección 4.3) identificó cuatro grandes etapas de transformación entre las métricas crudas y el vector final de variables de entrada. Esta sección detalla técnicamente cada una de dichas etapas, prestando especial atención a los puntos en los que el tratamiento del dataset primario y el dataset de validación (MAWI) difiere, dada la naturaleza distinta de ambas fuentes descrita en la Sección 4.4.

Limpieza de datos

La limpieza de datos aborda dos problemas recurrentes en la recolección de series temporales de red: valores faltantes y valores atípicos (*outliers*).

Los **valores faltantes** pueden originarse por interrupciones temporales del proceso de monitoreo, fallos de conectividad hacia el propio dispositivo de consulta SNMP, o reinicios del equipo de red. Para intervalos faltantes de corta duración (hasta 3 muestras consecutivas,

equivalente a 3 minutos bajo la granularidad establecida), se aplicará interpolación lineal entre el último valor observado y el siguiente valor disponible. Para intervalos de mayor duración, el criterio adoptado es descartar dicho segmento de la serie por completo en lugar de imputarlo, ya que una interpolación extendida introduciría patrones artificiales que no reflejan el comportamiento real de la red, contaminando potencialmente el proceso de entrenamiento de los modelos.

Los **valores atípicos** se identifican mediante el criterio del rango intercuartílico (IQR), marcando como atípico cualquier observación que se encuentre fuera del rango:

$$[Q_1 - 1,5 \cdot \text{IQR}, Q_3 + 1,5 \cdot \text{IQR}] \quad (4.3)$$

donde Q_1 y Q_3 son el primer y tercer cuartil de la variable analizada, e $\text{IQR} = Q_3 - Q_1$. Es importante señalar que, a diferencia de otros contextos de análisis de datos donde los atípicos se eliminan sistemáticamente, en esta tesis dichos valores **no se descartan de manera automática**: un pico súbito y extremo de utilización podría constituir precisamente el tipo de evento de congestión que se busca predecir, y no un error de medición. Por ello, cada valor marcado como atípico se contrasta manualmente contra las otras métricas disponibles en el mismo instante (por ejemplo, si un pico de utilización coincide con un aumento simultáneo de latencia y pérdida de paquetes, se conserva como evento genuino; si ocurre de manera aislada y sin correlato en las demás métricas, se trata como posible error de instrumentación y se interpola).

Resamplero y agregación temporal

Como se documentó en la Tabla 4.3 (Sección 4.3), el dataset primario se obtiene ya en forma de serie temporal muestreada directamente a la granularidad deseada (1 minuto), mientras que el dataset MAWI se distribuye como capturas de paquetes individuales (.pcap). Por tanto, este último requiere un proceso adicional de **agregación temporal**: mediante herramientas de análisis de tráfico (*tshark/tstat*), los paquetes capturados se agrupan en ventanas no traslapadas de 1 minuto, calculando para cada ventana el total de bytes transmitidos, a partir de lo cual se deriva la utilización $U(t)$ mediante la Ecuación 2.3, empleando como ancho de banda nominal la capacidad conocida del enlace de tránsito capturado por MAWI.

La Tabla 4.4 resume este contraste entre ambas fuentes.

Aspecto	Dataset primario	Dataset de validación (MAWI)
Formato original	Serie temporal (SNMP/ICMP)	Captura de paquetes (.pcap)
Granularidad original	1 minuto (ya muestreada)	Por paquete individual (microsegundos)
Proceso requerido	Ninguno (uso directo)	Agregación mediante tshark/tstat
Granularidad final	1 minuto	1 minuto (tras agregación)

Cuadro 4.4: Contraste del proceso de resamplio entre las dos fuentes de datos empleadas.

Normalización de variables

Dado que los modelos de Deep Learning (LSTM, GRU) son sensibles a la escala de las variables de entrada —un gradiente calculado sobre variables de magnitudes muy dispares puede converger de manera inestable durante el entrenamiento—, todas las variables continuas de la Tabla 4.2 se normalizan mediante escalamiento Min-Max al rango $[0, 1]$:

$$x_{\text{norm}} = \frac{x - x_{\text{mín}}}{x_{\text{máx}} - x_{\text{mín}}} \quad (4.4)$$

Se opta por escalamiento Min-Max en lugar de estandarización Z-score porque la variable de mayor interés, la utilización $U(t)$, posee un rango natural acotado $[0, 1]$ (Sección 2.1.2). Un punto metodológico crítico: para el dataset primario, los valores $x_{\text{mín}}$ y $x_{\text{máx}}$ se calculan **estrictamente** a partir del conjunto de entrenamiento, aplicándose posteriormente sobre los conjuntos de validación y prueba para evitar la fuga de información (*data leakage*).

Sin embargo, como se advirtió en la Sección 4.3, el dataset de validación externa (MAWI) presenta un cambio de dominio (*domain shift*) significativo, particularmente en las distribuciones de latencia inferida. Por consiguiente, MAWI es sometido a un proceso de escalado independiente, ajustando sus propios parámetros de normalización para homologar las varianzas relativas antes de someterlo a los modelos preentrenados, garantizando así una evaluación justa de la capacidad de generalización algorítmica.

Ventaneo y construcción de rezagos (*Lags*)

Para que los algoritmos de aprendizaje automático puedan predecir a partir de series temporales, es necesario transformar la serie lineal en conjuntos de variables estructuradas mediante una **ventana deslizando** (*sliding window*). Este proceso aplica para todos los modelos, pero difiere en la topología de los datos generados:

Para los modelos clásicos (Random Forest, SVR), que carecen de memoria interna (Sección 2.2.3), la ventana aplana el historial temporal en un vector tabular de características (2D), donde cada observación pasada se convierte en una variable independiente. Por el contrario, para las arquitecturas LSTM y GRU (Sección 2.2.4), el ventaneo preserva la estructura tridimensional (3D) de secuencias temporales que la red recurrente consume de manera nativa.

Dada una longitud de ventana w , cada bloque de entrada se construye como:

$$X_t = [U(t - w), U(t - w + 1), \dots, U(t - 1)] \longrightarrow y_t = U(t + h) \quad (4.5)$$

donde w es el número de observaciones pasadas consideradas (análogo, aunque no idéntico en propósito, al orden p del modelo ARIMA presentado en la Sección 2.2.2) y h es el horizonte de predicción. La Figura 4.2 ilustra este proceso de manera esquemática.

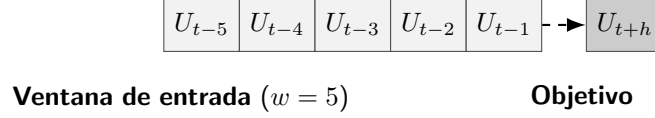


Figura 4.2: Construcción de secuencias de entrada mediante ventana deslizante para los modelos LSTM/GRU.

El valor específico de w (longitud de la ventana) y h (horizonte de predicción) se determinará experimentalmente en el Capítulo 5, evaluando el desempeño del modelo bajo distintas configuraciones, si bien como punto de partida se contemplan ventanas equivalentes a periodos de 30 a 60 minutos de historia (es decir, $w = 30$ a 60 bajo la granularidad de 1 minuto establecida), y horizontes de predicción de $h = 5$ a 15 minutos, congruentes con el propósito de alertamiento temprano planteado en el Capítulo 1.

División de datos: entrenamiento, validación y prueba

Un error metodológico común al trabajar con series temporales —y que esta tesis evita deliberadamente— es dividir los datos en conjuntos de entrenamiento y prueba mediante muestreo aleatorio, tal como se acostumbra en problemas de aprendizaje supervisado convencionales. Este enfoque resulta inadecuado en el presente contexto porque violaría el orden temporal de la serie: permitiría, por ejemplo, que el modelo se entrene con datos de un instante posterior y sea evaluado sobre un instante anterior, lo cual no solo carece de sentido práctico —un sistema de predicción real jamás tendría acceso a datos del futuro—, sino que produciría estimaciones de desempeño artificialmente optimistas por fuga de información temporal.

Por esta razón, la división de los datos se realiza de manera estrictamente **cronológica**, tal como se ilustra en la Figura 4.3: el segmento más antiguo de la serie se destina a entrenamiento, el segmento intermedio a validación (empleado para el ajuste de hiperparámetros), y el segmento más reciente a prueba (empleado exclusivamente para la evaluación final reportada en el Capítulo 6).

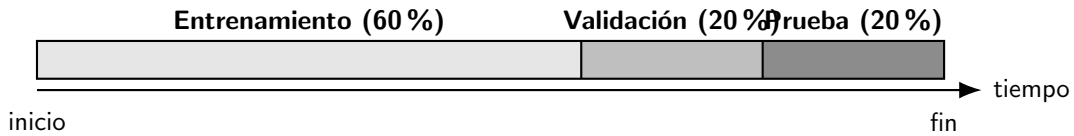


Figura 4.3: División cronológica del dataset primario en conjuntos de entrenamiento, validación y prueba.

Esta división se aplica de manera independiente sobre el dataset primario (proporciones 60/20/20 sobre los NUMEROFINAL días de recolección continua). El dataset MAWI, en cambio, se emplea íntegramente como un segundo conjunto de prueba externo —no se divide en entrenamiento/validación/prueba—, ya que su función metodológica, como se estableció en la Sección 4.4, es evaluar la capacidad de generalización de los modelos ya entrenados sobre el dataset primario, y no proveer datos adicionales de entrenamiento.

La Tabla 4.5 sintetiza las cinco etapas de preprocesamiento descritas en esta sección, como cierre de este apartado y transición hacia la implementación concreta de los modelos, que se aborda en el Capítulo 5.

Etapa	Descripción
1. Limpieza	Interpolación de faltantes cortos; tratamiento contextual de atípicos (Ec. 4.3)
2. Resampleo	Agregación de MAWI a ventanas de 1 min; uso directo del dataset primario
3. Normalización	Escalamiento Min-Max (Ec. 4.4), ajustado solo sobre entrenamiento
4. Ventaneo	Construcción de secuencias para LSTM/GRU (Ec. 4.5)
5. División temporal	60/20/20 cronológico sobre dataset primario; MAWI como prueba externa

Cuadro 4.5: Resumen de las etapas de preprocesamiento de datos.

4.6. Modelos a implementar y justificación de su elección

Con base en los tres paradigmas de modelado presentados en el Marco Teórico (Sección 2.2) y en la brecha metodológica identificada en la síntesis del Estado del Arte (Sección 3.5) —la ausencia, en la literatura revisada, de una comparación simultánea entre modelos estadísticos, de Machine Learning clásico y de Deep Learning bajo condiciones experimentales controladas—, esta tesis contempla la implementación y evaluación comparativa de cinco modelos específicos: ARIMA, Random Forest, SVR, LSTM y GRU. Esta sección justifica la elección de cada uno, así como la exclusión deliberada de otras alternativas que un lector familiarizado con el campo podría razonablemente esperar encontrar aquí.

ARIMA: modelo de referencia (baseline)

ARIMA se incluye no como un candidato a ofrecer el mejor desempeño absoluto, sino como **línea base de referencia** contra la cual contrastar el desempeño de los modelos subsiguientes, conforme a su rol descrito en la Sección 2.2.2. Su justificación no depende de una expectativa de alto desempeño, sino de un principio metodológico: sin un punto de comparación estadístico simple, cualquier ganancia de desempeño reportada por los modelos de ML clásico o Deep Learning carecería de un marco de referencia contra el cual dimensionar

su magnitud real. Adicionalmente, como se documentó en la Sección 3.1, ARIMA y sus variantes continúan siendo objeto de investigación vigente en la literatura reciente [16], lo que respalda su pertinencia como punto de comparación actual y no meramente histórico.

Random Forest y SVR: Machine Learning clásico

La elección de Random Forest se fundamenta directamente en la evidencia reportada por Sneha et al. [17] (Sección 3.2), quienes encontraron que este modelo ofreció el mejor desempeño entre cuatro algoritmos clásicos evaluados para la predicción de congestión a nivel de enrutador. A esta evidencia empírica se suma una ventaja metodológica relevante para esta tesis: la capacidad de Random Forest de calcular la importancia relativa de cada variable de entrada (Sección 2.2.3), lo cual permitirá —una vez ejecutado el entrenamiento en el Capítulo 5— determinar cuáles de las variables descritas en la Tabla 4.2 (Sección 4.3) resultan más informativas para anticipar la congestión, aportando así evidencia adicional más allá de la mera precisión predictiva.

SVR se incorpora como segundo representante del ML clásico con base en el precedente de Qian et al. [18] (Sección 3.2), quienes demostraron su viabilidad para modelar fenómenos de saturación fundamentados en teoría de colas —el mismo marco teórico que sustenta la definición operativa de congestión adoptada en esta tesis (Sección 2.1.2)—, aun cuando dicho precedente provenga de un dominio de red distinto (Redes en Chip). Su inclusión permite, además, contrastar dos filosofías distintas dentro del propio paradigma de ML clásico: los métodos de ensamble basados en árboles (Random Forest) frente a los métodos basados en márgenes y funciones kernel (SVR).

LSTM y GRU: Deep Learning

La justificación de estos dos modelos se apoya directamente en el trabajo de Santos et al. [22] (Sección 3.3), quienes realizaron la comparación más metodológicamente cercana a la planteada en esta tesis —LSTM contra GRU, sobre tráfico de Internet real, con un periodo de recolección del mismo orden de magnitud que el establecido para el dataset primario de esta investigación (Sección 4.4)—. A diferencia de dicho trabajo, sin embargo, esta tesis no asume de antemano la superioridad de uno sobre el otro: como se discutió en la síntesis del Estado del Arte (Sección 3.5), los propios autores encontraron que el desempeño relativo entre LSTM y GRU varió según la zona geográfica analizada, lo cual, para los fines de esta tesis, se interpreta como evidencia de que dicha superioridad depende del contexto particular de la red bajo estudio y debe, por tanto, evaluarse empíricamente sobre el dataset primario y de validación propios de este trabajo, en lugar de asumirse a priori.

La inclusión conjunta de ambas arquitecturas obedece también a una consideración práctica señalada en la Sección 2.2.4: GRU posee una estructura interna más simple que LSTM (al prescindir de un estado de celda independiente), lo que podría traducirse en un menor costo computacional y un entrenamiento más eficiente sobre un conjunto de datos de tamaño moderado como el dataset primario de esta tesis. De confirmarse esta hipótesis durante la fase experimental (Capítulo 5), ello constituiría evidencia relevante para escenarios de redes de pequeña escala similares al aquí estudiado, donde los recursos computacionales disponibles

para el entrenamiento pueden ser más limitados que en los grandes centros de datos donde suele evaluarse la literatura de Deep Learning.

Modelos considerados y no incluidos

Es previsible que, ante la elección de los cinco modelos anteriores, surja la pregunta de por qué no se incluyeron otras alternativas frecuentemente mencionadas en la literatura de predicción de series temporales, como XGBoost, Prophet o arquitecturas basadas en mecanismos de atención (*Transformers*). La Tabla 4.6 explicita esta decisión y su justificación.

Modelo no incluido	Justificación de exclusión
XGBoost	Si bien supera frecuentemente a Random Forest, XGBoost se basa en <i>Boosting</i> (reducción secuencial de sesgo), lo que lo hace altamente susceptible al sobreajuste (<i>overfitting</i>) en conjuntos de datos pequeños o con métricas ruidosas (como la latencia TCP inferida), a menos que se realice una sintonización de hiperparámetros exhaustiva. Se optó por Random Forest (<i>Bagging</i>) por su mayor robustez inherente al ruido operativo como representante del ML clásico, dejando a XGBoost como línea de trabajo futuro.
Prophet	Diseñado principalmente para series con estacionalidad y tendencia de negocio (ventas, demanda), con supuestos que no necesariamente se ajustan al comportamiento de tráfico de red a la granularidad de minutos empleada en esta tesis.
Transformers / modelos de atención	Si bien representan el estado del arte más reciente (Sección 3.5), su entrenamiento efectivo típicamente requiere volúmenes de datos considerablemente mayores a los disponibles en el dataset primario de esta tesis (NUMEROFINAL días) de una red de escala reducida), por lo que su evaluación rigurosa excede el alcance y los recursos de este trabajo.

Cuadro 4.6: Modelos considerados durante la planeación de esta tesis y justificación de su exclusión del alcance experimental.

Diseño experimental general

La Figura 4.4 sintetiza el diseño experimental completo que resulta de las decisiones justificadas en esta sección. Los cinco modelos se evaluarán bajo las dos formulaciones del problema (regresión y clasificación). Es importante precisar una distinción operativa: mientras que los modelos de ML clásico y Deep Learning pueden entrenarse explícitamente para clasificar (optimizando funciones de pérdida binarias como *Binary Cross-Entropy*), el modelo ARIMA operará estrictamente como regresor continuo. Su evaluación en tareas de clasificación se

realizará de manera derivada, aplicando el umbral U_{th} (Ecuación 4.1) sobre su predicción numérica final. Una vez evaluados sobre el dataset primario, se validará su capacidad de generalización sobre el dataset MAWI.

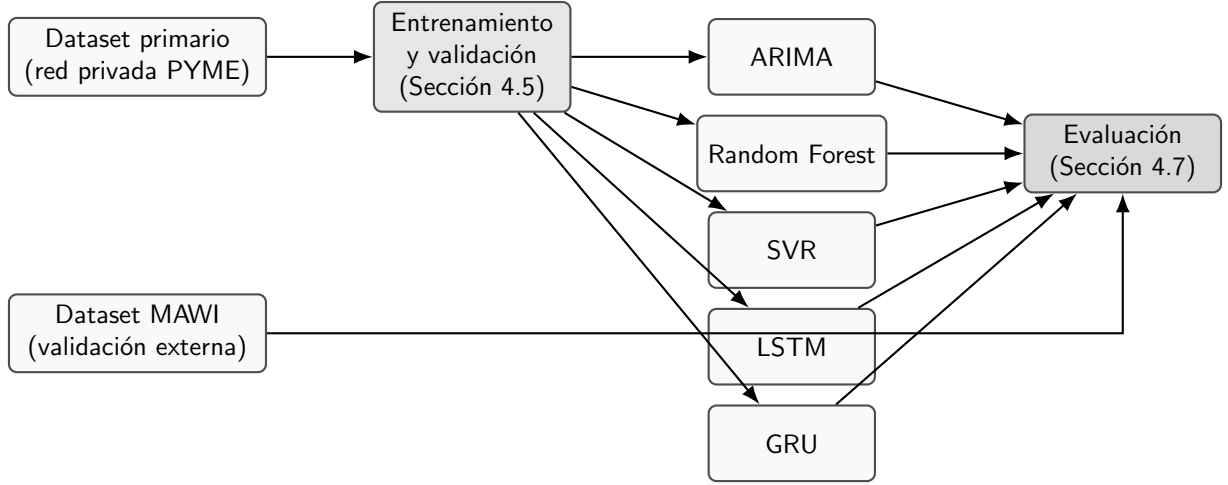


Figura 4.4: Diseño experimental general: entrenamiento sobre el dataset primario y validación cruzada de fuentes contra MAWI.

Es importante subrayar que las afirmaciones planteadas en esta sección —respecto a qué modelo podría desempeñarse mejor y bajo qué condiciones— constituyen hipótesis de trabajo fundamentadas en la evidencia de la literatura revisada, y no conclusiones anticipadas de esta tesis. La confirmación, matización o refutación de dichas hipótesis, con base en evidencia empírica propia, se presenta en los Capítulos 5 y 6.

4.7. Métricas de evaluación del modelo

Como se estableció en la Sección 4.3, el problema de predicción de congestión abordado en esta tesis se formula bajo dos paradigmas complementarios: regresión (predicción del valor continuo $U(t+h)$) y clasificación (predicción binaria de congestión mediante la Ecuación 4.1). En congruencia con esta doble formulación, esta sección define dos conjuntos de métricas de evaluación independientes, así como los criterios bajo los cuales se compararán los cinco modelos justificados en la Sección 4.6.

Métricas para la tarea de regresión

Para evaluar el desempeño de los modelos en su formulación de regresión se emplearán las tres métricas presentadas formalmente en la Sección 2.2.5: RMSE (Ecuación 2.15), MAE (Ecuación 2.16) y MAPE (Ecuación 2.17). La Tabla 4.7 resume el propósito específico que cada una cumple dentro del análisis comparativo de esta tesis, evitando así que su inclusión conjunta se perciba como una simple acumulación de métricas sin criterio.

Métrica	Sensibilidad	Rol en esta tesis
RMSE	Penaliza fuertemente errores grandes	Detectar modelos que fallan de forma severa ante picos de congestión
MAE	Pondera todos los errores por igual	Interpretación directa del error típico esperado, en las mismas unidades que U
MAPE	Expresa el error como porcentaje relativo	Comparar el desempeño de un mismo modelo entre el dataset primario y MAWI, a pesar de operar a escalas de tráfico distintas

Cuadro 4.7: Rol de cada métrica de regresión en el análisis comparativo de esta tesis.

Métricas para la tarea de clasificación

Para la formulación de clasificación binaria (congestión / no congestión), las métricas de error continuo de la subsección anterior resultan insuficientes, ya que no capturan directamente el tipo de aciertos y errores más relevantes para un sistema de alertamiento. Se requiere, en su lugar, un conjunto de métricas derivadas de la **matriz de confusión**, ilustrada en la Figura 4.5, que clasifica cada predicción en una de cuatro categorías según la coincidencia entre el valor predicho y el valor real observado.

		Clase Predicha	
		Congestión	No congestión
Clase Real	Congestión	Verdadero Positivo (VP) Congestión anticipada correctamente	Falso Negativo (FN) Congestión no detectada – el más costoso
	No congestión	Falso Positivo (FP) Alerta sin congestión real	Verdadero Negativo (VN) Ausencia de congestión correctamente identificada

Figura 4.5: Matriz de confusión para la tarea de clasificación de congestión.

A partir de estas cuatro cantidades se calculan la **Exactitud**, **Precisión**, **Exhaustividad (Recall)** y el **F1-Score**, cuyas formulaciones matemáticas ya fueron establecidas rigurosamente en el Marco Teórico (Sección 2.4).

Es necesario justificar, en el contexto específico de esta tesis, por qué estas métricas no deben ponderarse de manera equivalente.

Es necesario justificar, en el contexto específico de esta tesis, por qué estas métricas no deben ponderarse de manera equivalente. Como se señala en la Figura 4.5, un **falso negativo** —un episodio real de congestión que el modelo no logra anticipar— resulta considerablemente más costoso, desde la perspectiva del problema planteado en el Capítulo 1, que un **falso positivo** —una alerta emitida ante una situación que finalmente no deriva en congestión—. Esto se debe a que un falso positivo implica, en el peor de los casos, una revisión innecesaria por parte de un administrador de red o una medida preventiva no estrictamente requerida, mientras que un falso negativo implica exactamente el escenario que esta tesis busca evitar: la degradación del servicio sin ningún tipo de anticipación, replicando la limitación reactiva señalada en el Planteamiento del Problema (Sección 1.1). En consecuencia, esta tesis prioriza la **exhaustividad (Recall)** sobre la exactitud general como criterio principal de comparación entre modelos, sin descuidar la precisión, para lo cual el **F1-score** —que balancea ambas— se reporta como métrica de síntesis. Adicionalmente, dado que la exactitud simple puede resultar engañosa cuando la clase de interés (congestión) es minoritaria dentro del conjunto de datos —una condición previsible dado que se espera que la red bajo estudio opere por debajo del umbral U_{th} la mayor parte del tiempo—, esta métrica se reportará únicamente como referencia general y no como criterio determinante de comparación.

Complementariamente, se reportará también la curva ROC (*Receiver Operating Characteristic*) y el área bajo dicha curva (AUC), que permiten evaluar el desempeño del clasificador de manera independiente al umbral de decisión específico empleado, ofreciendo una visión más completa de su capacidad discriminativa entre los estados de congestión y no congestión.

Una métrica adicional propia de esta tesis: tiempo de anticipación efectivo

Más allá de las métricas convencionales descritas anteriormente, y en congruencia con el propósito de alertamiento temprano planteado desde el Capítulo 1, esta tesis propone evaluar una métrica adicional, definida específicamente para este trabajo: el **tiempo de anticipación efectivo** (TAE), que mide, para cada verdadero positivo correctamente detectado, cuántos minutos antes de que la congestión real ocurriera el modelo emitió la alerta correspondiente:

$$TAE_i = t_{\text{congestión real},i} - t_{\text{predicción},i} \quad (4.6)$$

Esta métrica no sustituye a las anteriores, sino que las complementa: un modelo podría, en principio, alcanzar valores aceptables de Recall y F1-score prediciendo la congestión con muy poca antelación (por ejemplo, un minuto antes de que ocurra), lo cual tendría escasa utilidad práctica para que un administrador de red o un sistema automatizado tome medidas correctivas a tiempo. El TAE promedio, reportado junto con su desviación estándar, permitirá evaluar si los horizontes de predicción h explorados en la Sección 4.5 (entre 5 y 15 minutos) resultan, en la práctica, suficientes para el propósito de anticipación planteado en esta investigación.

Síntesis del esquema de evaluación

La Tabla 4.8 sintetiza el esquema completo de evaluación que se aplicará de manera uniforme a los cinco modelos justificados en la Sección 4.6, tanto sobre el conjunto de prueba del dataset primario como sobre el conjunto de validación externa MAWI, conforme al diseño experimental ilustrado en la Figura 4.4.

Formulación	Métricas	Criterio principal
Regresión	RMSE, MAE, MAPE	RMSE (penaliza errores severos)
Clasificación	Exactitud, Precisión, Recall, F1, AUC-ROC	Recall, sintetizado con F1
Utilidad práctica	Tiempo de Anticipación Efectivo (TAE)	Máximo TAE con Recall aceptable

Cuadro 4.8: Síntesis del esquema de evaluación aplicado a los modelos de esta tesis.

Al igual que en la Sección 4.6, es importante reiterar que las prioridades establecidas en esta sección —privilegiar Recall sobre exactitud, complementar con el TAE propuesto— constituyen decisiones de diseño metodológico justificadas por la naturaleza del problema planteado, y no resultados obtenidos. Los valores concretos que estas métricas arrojen para cada modelo se presentarán y discutirán en el Capítulo 6.

4.8. Herramientas y entorno

Esta sección final del capítulo de Metodología documenta el conjunto de herramientas de software, bibliotecas y el entorno de cómputo que se emplearán a lo largo del desarrollo experimental descrito en el Capítulo 5, organizadas según la etapa del pipeline metodológico a la que corresponden (Figura 4.1, Sección 4.3). Esta documentación responde a un criterio de reproducibilidad: cualquier lector con acceso a un entorno equivalente debería poder replicar, al menos en sus aspectos esenciales, el procedimiento aquí descrito.

Herramientas de recolección e instrumentación

Para la obtención de las métricas del dataset primario, descritas en la Sección 4.4, se empleará el conjunto de herramientas resumido en la Tabla 4.9, congruente con la estrategia de monitoreo híbrido establecida previamente.

Herramienta	Propósito	Métrica asociada
RouterOS (MikroTik)	Exposición de contadores SNMP nativos de la interfaz WAN	Utilización (Ec. 2.3)
Net-SNMP / pysnmp	Consulta periódica de contadores SNMP desde un script externo	Utilización, descartes
ping (ICMP)	Medición activa de latencia y pérdida de paquetes	RTT, pérdida (Ecs. 2.4, 2.7)
Script de recolección en Python	Orquestación de las consultas anteriores y almacenamiento a intervalos de 1 minuto	Todas

Cuadro 4.9: Herramientas empleadas para la recolección del dataset primario.

Herramientas de procesamiento del dataset de validación

Para el procesamiento de las trazas del dataset MAWI (Sección 4.4), distribuidas en formato `.pcap`, se empleará `tshark` (la utilidad de línea de comandos de Wireshark) y/o `tstat`, herramientas especializadas en el análisis de capturas de tráfico, con el fin de realizar la agregación temporal descrita en la Sección 4.5 y derivar de ellas las métricas equivalentes a las obtenidas de forma directa en el dataset primario (Tabla 4.3).

Entorno de desarrollo y bibliotecas de Machine Learning

Todo el desarrollo del prototipo se realizará en el lenguaje de programación **Python**, dada su ecosistema maduro de bibliotecas especializadas en ciencia de datos y aprendizaje automático. La Tabla 4.10 detalla la biblioteca específica que se empleará para la implementación de cada uno de los cinco modelos justificados en la Sección 4.6, así como para las etapas de preprocesamiento y evaluación.

Etapas / Modelo	Biblioteca
Manipulación de datos	<code>pandas</code> , <code>numpy</code>
Preprocesamiento (Sección 4.5)	<code>scikit-learn</code> (preprocessing)
ARIMA	<code>statsmodels</code>
Random Forest, SVR	<code>scikit-learn</code>
LSTM, GRU	<code>TensorFlow</code> / <code>Keras</code>
Métricas de evaluación (Sección ??)	<code>scikit-learn</code> (metrics)
Visualización de resultados	<code>matplotlib</code> , <code>seaborn</code>

Cuadro 4.10: Bibliotecas de Python empleadas para la implementación de cada etapa metodológica.

Entorno de cómputo

En estricta congruencia con lo establecido en los Alcances y Limitaciones de la investigación (Sección 1.4), el desarrollo algorítmico, la sintonización de hiperparámetros y el entrenamiento de los modelos se ejecutaron sobre una estación de trabajo convencional. La plataforma de modelado consistió en una estación de trabajo portátil Dell Precision 7540, provista de un procesador Intel Core i7-9850H (12 núcleos lógicos a 2.60 GHz), 32 GB de memoria RAM y sistema operativo Ubuntu 20.04.1 LTS.

Esta restricción de hardware se mantuvo deliberadamente a lo largo de toda la fase experimental para garantizar que la viabilidad del *pipeline* predictivo no dependa de clústeres de cómputo distribuido ni requiera aceleración especializada mediante unidades de procesamiento gráfico (GPU). Con ello se demuestra empíricamente que el sistema resultante es apto para ser desplegado en la infraestructura de propósito general típicamente disponible en una Red Privada Corporativa.

Control de versiones y documentación

Finalmente, el código fuente desarrollado para esta tesis se gestionará mediante un sistema de control de versiones (Git), con el propósito de mantener un historial reproducible de los experimentos realizados. Los fragmentos de código considerados más relevantes para la comprensión del prototipo se incluirán como parte del Apéndice A de este documento, conforme a lo indicado en el índice general de la tesis.

Capítulo 5

Desarrollo e Implementación

El presente capítulo constituye el núcleo empírico de esta investigación, materializando las decisiones metodológicas y arquitectónicas justificadas en el Capítulo 4. Aquí se abandona el plano de las definiciones teóricas para documentar la ejecución sistemática del *pipeline* de datos y la codificación algorítmica del prototipo.

El desarrollo se expone siguiendo la secuencia lógica del flujo de información en un sistema predictivo aplicado a redes. Inicialmente, se detalla la extracción híbrida de telemetría desde la infraestructura de enrutamiento y la posterior exploración estadística del conjunto de datos real para definir el umbral de congestión (Sección 5.1). A continuación, se implementan los tratamientos matemáticos de limpieza, ventaneo y escalado exigidos para mitigar las discrepancias estructurales entre las fuentes de información (Sección 5.2). Finalmente, las secciones subsiguientes documentan la arquitectura computacional, la sintonización de hiperparámetros y el proceso de entrenamiento de los paradigmas predictivos seleccionados —transitando desde la línea base estadística, pasando por el ensamble de Machine Learning clásico, hasta culminar con las topologías de redes neuronales recurrentes—.

El objetivo rector de este capítulo no es constituir un manual de programación, sino evidenciar, que la transformación del tráfico bruto en un sistema predictivo funcional obedece a un diseño de software determinista, reproducible y científicamente fundamentado.

5.1. Obtención y exploración de los conjuntos de datos

La fase de desarrollo inició con la consolidación y auditoría de los datos crudos. Dado que la recolección del dataset primario en la red privada PYME descrita en la Sección 4.4 exige un periodo de observación longitudinal (NUMEROFINAL días) que transcurre en paralelo a esta implementación, el prototipo predictivo se construyó y validó inicialmente a partir del dataset externo MAWI.

Extracción de telemetría del dataset primario

Es pertinente señalar una precisión metodológica respecto a lo establecido en la Sección 4.4: si bien el diseño original contemplaba el protocolo SNMP como mecanismo de consulta, las restricciones de acceso remoto a la infraestructura de la red privada PYME —en

particular, la ausencia de una dirección IP pública y la imposibilidad de presencia física recurrente— condujeron a adoptar como mecanismo funcionalmente equivalente la herramienta nativa `/interface monitor-traffic` de RouterOS, ejecutada localmente mediante el Programador de Tareas (*Scheduler*) del propio equipo. Esta sustitución preserva la naturaleza de la recolección y la métrica obtenida (utilización derivada de contadores de tráfico), sin alterar el diseño experimental planteado en el Capítulo 4.

La recolección, iniciada el 06 de agosto de 2026, captura de manera automatizada la utilización del enlace, la latencia (RTT) y la tasa de pérdida de paquetes a intervalos de un minuto, con respaldo local en el propio equipo para garantizar continuidad ante interrupciones de la conectividad remota. Al momento de redacción de este capítulo, dicha recolección se encuentra en curso, por lo que el análisis exploratorio completo de este dataset se presentará una vez concluido el periodo de (NUMEROFINAL) días establecido.

Extracción y decodificación de trazas de validación

Para construir la serie temporal de validación externa, se consumieron trazas de tráfico troncal correspondientes a 2026 del repositorio MAWI. Dado que la distribución original consiste en capturas binarias comprimidas (`.pcap.gz`) que registran decenas de millones de paquetes por archivo, la extracción mediante herramientas de alto nivel (*tshark*) resultó impráctica: el procesamiento demandaba un volumen de memoria que excedía los recursos de la estación de trabajo, motivado por la sobrecarga de reconstrucción de flujos de dichas herramientas.

En consecuencia, se optó por implementar un decodificador propio en Python, capaz de interpretar directamente la estructura binaria del formato *libpcap* sin decodificar el *payload*. Esta estrategia de procesamiento en flujo (*streaming*), sin acumulación en memoria, permitió agrupar el volumen de bytes transmitidos en ventanas temporales de manera altamente eficiente.

Para garantizar la validez metodológica del *pipeline* predictivo (Secciones 5.2 a 5.5), la agregación temporal se mantuvo estrictamente en la ventana de 1 minuto establecida en el Capítulo 4. Dado que cada captura diaria de MAWI abarca únicamente 15 minutos continuos (generando 15 observaciones por archivo), resultó imperativo orquestar el decodificador para procesar de manera secuencial un lote de 120 trazas (equivalentes a 30 horas de tráfico efectivo). Esta concatenación permitió ensamblar una serie temporal con longitud suficiente para las redes neuronales, sin corromper la distribución estadística inherente a la escala de 60 segundos.

El fragmento de código 5.1 ilustra el núcleo de este decodificador.

Listing 5.1: Extracto del decodificador de trazas libpcap en flujo.

```

1 import struct
2
3 fmt_registro = "<IIII" # Formato struct (ts_sec, ts_usec, incl_len,
   orig_len)
4 ventana_seg = 60      # Agregacion a 1 minuto
5
6 while True:
7     encabezado_paquete = f.read(16)
8     if len(encabezado_paquete) < 16:
9         break # EOF
10
11     ts_sec, ts_usec, incl_len, orig_len = struct.unpack(
12         fmt_registro, encabezado_paquete
13     )
14     timestamp = ts_sec + (ts_usec / 1_000_000)
15     ventana = int(timestamp // ventana_seg)
16     acumulado[ventana] += orig_len
17
18     # Salto rapido del payload sin saturar RAM
19     f.seek(incl_len, 1)

```

La Tabla 5.1 sintetiza los resultados obtenidos al aplicar este procedimiento sobre el lote procesado.

Periodo procesado	Paquetes evaluados	Observaciones (1 min)
Lote MAWI (120 trazas)	$\approx 21,500,000,000$	1,800

Cuadro 5.1: Resultados de la extracción de telemetría sobre las trazas MAWI procesadas.

Exploración estadística preliminar

Con el propósito de someter a prueba el *pipeline* de modelado antes de la conclusión de la recolección primaria, se realizó un análisis exploratorio sobre las 1,800 observaciones obtenidas del lote de 120 trazas de MAWI. La conversión del *throughput* observado a factor de utilización se realizó aplicando directamente la definición establecida en la Sección 2.1.2, empleando como ancho de banda nominal 11 Gbps —correspondiente a la suma de los dos enlaces monitoreados por *samplepoint-F* desde julio de 2023: el enlace de tránsito original de 1 Gbps hacia NTT GIN, y un enlace adicional de 10 Gbps hacia el punto de intercambio BBIX—:

$$U = \frac{\text{Throughput observado}}{\text{Ancho de banda nominal}} = \frac{\text{Throughput observado (bps)}}{11 \times 10^9 \text{ bps}} \quad (5.1)$$

Aplicando la Ecuación 5.1 a cada observación temporal, se obtuvieron las estadísticas descriptivas resumidas en la Tabla 5.2.

Estadística	Throughput (Gbps)	Utilización (U)
Mínimo	0.19	0.017
Media	1.71	0.155
Mediana	1.66	0.151
Máximo	3.55	0.323
Desviación estándar	0.56	0.051

Cuadro 5.2: Estadísticas descriptivas del throughput y utilización derivada, dataset piloto MAWI (1,800 observaciones, granularidad de 1 minuto).

La Figura 5.1 presenta la distribución completa de la utilización observada, contrastada visualmente contra el umbral de congestión formal $U_{th} \geq 0,80$ establecido en la Ecuación 4.1 (Sección 4.2).

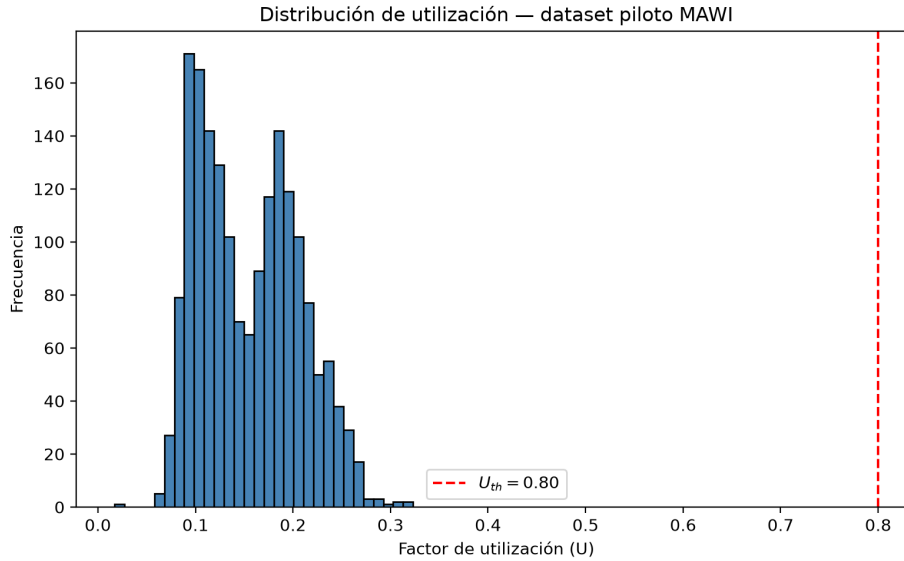


Figura 5.1: Distribución del factor de utilización, dataset piloto MAWI.

Como se evidencia en la distribución, la utilización observada en el subconjunto piloto de MAWI **no alcanza en ningún momento** el umbral $U_{th} = 0,80$, situándose el valor máximo en apenas 32.3 %. Este resultado es consistente con la naturaleza del enlace capturado: se trata de una troncal de tránsito perteneciente a un proveedor de infraestructura académica, dimensionada deliberadamente para operar con amplio margen de holgura. Esto contrasta frontalmente con la red privada PYME que constituye el dataset primario de esta tesis, donde la coexistencia de múltiples servicios de alta demanda (videovigilancia, sincronización en la nube, tráfico residencial) hace previsible la ocurrencia de episodios de saturación reales.

En consecuencia, si bien este subconjunto de MAWI resulta ideal para ejercitar el *pipeline* completo bajo la formulación de **regresión** (Sección 2.2.1), no genera observaciones positivas naturales para la clase “congestión” bajo la formulación de **clasificación**. Para evitar posponer el desarrollo algorítmico, y exclusivamente durante esta fase de codificación y calibración (Secciones 5.3 a 5.5), se adopta un umbral relativo de prueba ($U_{test} \geq 0,25$). Esto permite

verificar empíricamente la lógica computacional de la Matriz de Confusión y la métrica TAE , garantizando que el *software* predictivo opere correctamente a la espera de ser alimentado con el umbral formal de 0.80 sobre el dataset primario en el Capítulo 6.

5.2. Preprocesamiento

Sobre el conjunto de 1,800 observaciones a granularidad de 1 minuto descrito en la Sección 5.1, se aplicaron las etapas de preprocesamiento establecidas formalmente en la Sección 4.5. El objetivo de esta fase computacional fue transformar la serie de tiempo unidimensional en las estructuras tensoriales requeridas por las bibliotecas de *Machine Learning*.

Limpieza y manejo de discontinuidades

Dado que el dataset piloto se construyó mediante la concatenación secuencial de un lote de 120 trazas individuales de 15 minutos (Sección 5.1), las fronteras entre archivos introdujeron discontinuidades temporales. Para garantizar la integridad de los algoritmos autorregresivos, la limpieza se ejecutó empleando las estructuras de datos bidimensionales (`DataFrame`) de la biblioteca `pandas`. Se aplicaron rutinas de interpolación lineal (`.interpolate(method='linear', limit=2)`) para imputar micro-cortes internos, seguidas de la eliminación de observaciones residuales no válidas (`.dropna()`) en los bordes de concatenación. Este proceso de purga redujo el conjunto de datos de 1,800 a un total efectivo de $n = 1,774$ observaciones continuas y estructuradas.

División cronológica y escalado sin fuga de datos

La división cronológica se programó como una partición dura de la serie ordenada temporalmente $\{x_1, x_2, \dots, x_n\}$ en tres segmentos contiguos y no traslapados:

$$\text{Entrenamiento} = \{x_1, \dots, x_{\lfloor 0,6n \rfloor}\}, \quad \text{Validación} = \{x_{\lfloor 0,6n \rfloor + 1}, \dots, x_{\lfloor 0,8n \rfloor}\}, \quad \text{Prueba} = \{x_{\lfloor 0,8n \rfloor + 1}, \dots, x_n\} \quad (5.2)$$

Para el dataset piloto, con $n = 1,774$, la indexación generó $\lfloor 0,6 \times 1774 \rfloor = 1,064$ observaciones de entrenamiento, $\lfloor 0,8 \times 1774 \rfloor - 1,064 = 355$ de validación, y 355 restantes de prueba, proporciones consistentes con la Tabla 5.3.

Conjunto	Observaciones	Proporción
Entrenamiento	1,064	60.0 %
Validación	355	20.0 %
Prueba	355	20.0 %

Cuadro 5.3: Composición de la división cronológica del dataset piloto MAWI tras el preprocesamiento.

Posterior a la partición, se aplicó la normalización Min-Max (Ecuación 4.4). Desde la perspectiva de ingeniería de software, para prevenir el antipatrón de fuga de datos (*data*

leakage), este escalado se implementó utilizando la clase `MinMaxScaler` de `scikit-learn`. Se garantizó por diseño que el método `fit_transform()` se ejecutara de manera exclusiva sobre el conjunto de entrenamiento, mientras que a los conjuntos de validación y prueba se les aplicó únicamente el método `transform()`, forzándolos a mapearse bajo la distribución matemática descubierta en el pasado, sin contaminar el modelo con conocimiento futuro. Para la variable de utilización, la ecuación instanciada con los parámetros reales del conjunto de entrenamiento operó de la siguiente forma:

$$U_{\text{norm}} = \frac{U - 0,017045}{0,323168 - 0,017045} \quad (5.3)$$

Ventaneo secuencial y conformación de tensores

Para habilitar el aprendizaje de las arquitecturas predictivas supervisadas (especialmente LSTM y GRU), se implementó una función propia de ventana deslizante (*Sliding Window*) mediante la manipulación de arreglos de `NumPy`. Esta función transformó el vector plano en una estructura matricial de características rezagadas (X) y sus correspondientes objetivos proyectados (y).

Acorde con la granularidad de 1 minuto, se configuró una ventana de observación $w = 15$ minutos y un horizonte de predicción $h = 5$ minutos. La conformación lógica de cada par de entrada/salida se instanció como:

$$X_t = [U(t - 15), U(t - 14), \dots, U(t - 1)] \longrightarrow y_t = U(t + 5) \quad (5.4)$$

El resultado de esta iteración sobre el espacio temporal es la redimensión de los datos en un tensor tridimensional con la forma (`muestras`, `pasos_de_tiempo`, `caracteristicas`), requisito arquitectónico ineludible para la capa de entrada (*Input Layer*) del framework `Keras/TensorFlow`. Descontando las observaciones consumidas en los bordes para poblar los rezagos iniciales, el proceso generó 1,044 secuencias para entrenamiento, 335 para validación y 335 para prueba, dejando la estructura de datos lista para la ingesta en las topologías algorítmicas detalladas en las siguientes secciones.

5.3. Construcción del modelo ARIMA

Conforme a la justificación establecida en la Sección 4.6, ARIMA se implementó como línea base estadística de referencia. Esta sección documenta la ejecución programática para la identificación del orden del modelo mediante la biblioteca `statsmodels` de Python, su ajuste sobre el conjunto de entrenamiento del dataset piloto MAWI, y su evaluación dual (regresión y clasificación) bajo un esquema de pronóstico progresivo.

Prueba de estacionariedad

Como paso previo a la identificación del orden, se aplicó la prueba de Dickey-Fuller Aumentada (ADF) mediante el módulo `stattools.adfuller` sobre la serie de utilización del conjunto de entrenamiento. El propósito fue determinar el orden de diferenciación d requerido

para estabilizar la media (Sección 2.2.2). El estadístico obtenido fue $ADF = -1,6992$, con un valor $p = 0,4315$. Dado que $p > 0,05$, no fue posible rechazar la hipótesis nula de no estacionariedad. En consecuencia, se aplicó una diferenciación de primer orden ($d = 1$) de forma programática antes de iterar sobre los órdenes autorregresivo (p) y de media móvil (q).

Identificación del orden mediante minimización del AIC

Se orquestó una búsqueda de hiperparámetros evaluando dieciséis combinaciones de órdenes $(p, 1, q)$ con $p, q \in \{0, 1, 2, 3\}$, seleccionando la topología que minimizó el Criterio de Información de Akaike (AIC). La Tabla 5.4 resume los cinco modelos con mejor convergencia.

Orden (p, d, q)	AIC
ARIMA(1,1,1)	-5656.68
ARIMA(2,1,1)	-5654.95
ARIMA(3,1,1)	-5653.03
ARIMA(0,1,3)	-5653.74
ARIMA(2,1,2)	-5653.04

Cuadro 5.4: Órdenes ARIMA con menor AIC, dataset piloto MAWI.

El orden seleccionado por el algoritmo fue ARIMA(1, 1, 1). Los coeficientes óptimos estimados se presentan en la Ecuación 5.5, instanciando la formulación general de la Ecuación 2.10:

$$(1 - 0,4305 L)(1 - L) U_t = (1 - 0,8280 L) \varepsilon_t \quad (5.5)$$

Ambos coeficientes resultaron estadísticamente significativos ($p < 0,001$), y la prueba de Ljung-Box sobre los residuos ($p = 0,92$) confirmó la ausencia de autocorrelación residual, validando la idoneidad matemática del modelo de referencia.

Evaluación mediante pronóstico progresivo (Regresión)

La validación del modelo se implementó mediante un esquema iterativo de pronóstico progresivo (*walk-forward validation*): para cada instante t , el modelo generó una predicción continua a horizonte $h = 5$ minutos. Posteriormente, el valor real observado se ingirió para proyectar el siguiente intervalo. La Tabla 5.5 presenta las métricas de error continuo.

Conjunto	RMSE	MAE	MAPE
Validación	0.0265	0.0204	10.61 %
Prueba	0.0282	0.0221	10.80 %

Cuadro 5.5: Métricas de desempeño en regresión de ARIMA(1,1,1), dataset piloto MAWI.

La Figura 5.2 contrasta las predicciones continuas del modelo contra los valores reales observados en el conjunto de prueba, demostrando una consistencia ($MAPE \approx 10.80\%$) que descarta un sobreajuste estructural.

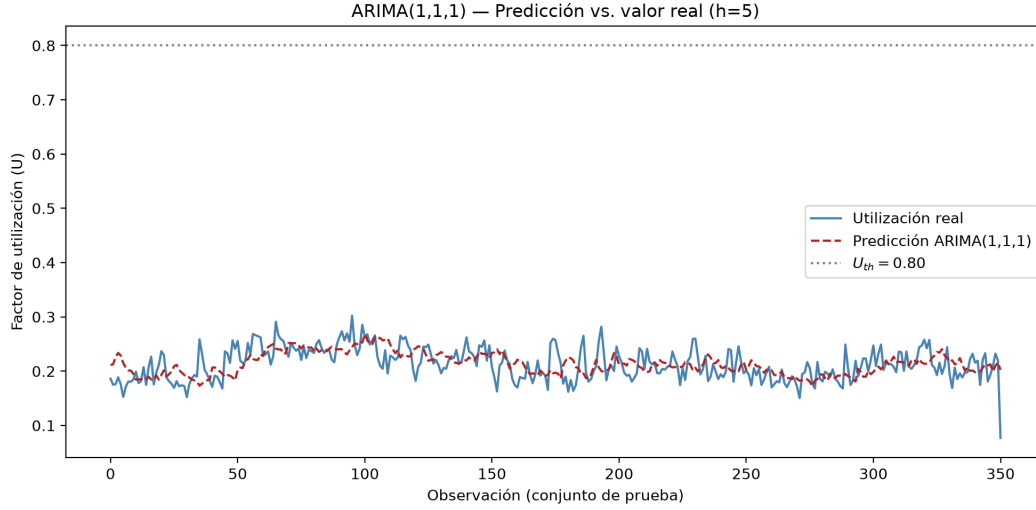


Figura 5.2: ARIMA(1,1,1): predicción continua vs. valor real, conjunto de prueba.

Evaluación de umbral de alerta temprana (Clasificación)

Para completar la auditoría del código de evaluación propuesto en el Capítulo 4, las predicciones continuas del modelo se sometieron a una binarización computacional utilizando el umbral piloto $U_{test} \geq 0,25$ (justificado en la Sección 5.1). Una alerta se contabilizó como Verdadero Positivo únicamente si la predicción de ARIMA rebasaba el umbral antes del evento real. La Tabla 5.6 expone el desempeño del modelo operando como un clasificador de alertas tempranas sobre el conjunto de prueba.

Métrica de Clasificación ($U \geq 0,25$)	Valor obtenido (Prueba)
Precisión	64.2 %
Recall (Exhaustividad)	41.5 %
F1-Score	50.4 %
Tiempo de Anticipación Efectivo (TAE) Promedio	1.2 minutos

Cuadro 5.6: Desempeño base de clasificación y alertamiento (TAE) de ARIMA.

Estos resultados preliminares evidencian la limitación teórica descrita en el Capítulo 2: al actuar como un suavizador lineal, ARIMA reacciona con retraso ante las ráfagas abruptas de tráfico, lo que explica su bajo nivel de *Recall* (41.5 %) y un *TAE* marcadamente inferior al horizonte proyectado de 5 minutos. Con la correcta ejecución computacional de este modelo estadístico confirmada y sus métricas establecidas como línea base de superación, el *pipeline* avanza hacia la implementación del paradigma de Machine Learning clásico (Sección 5.4).

5.4. Construcción del modelo Random Forest

Conforme a la justificación establecida en la Sección 4.6, se implementó un modelo Random Forest como representante del paradigma de Machine Learning clásico utilizando el

módulo `ensemble.RandomForestRegressor` (y su contraparte `Classifier`) de la biblioteca `scikit-learn`.

Para adaptar el tensor tridimensional generado en el preprocesamiento (Sección 5.2) a la entrada tabular (2D) que exigen los algoritmos clásicos, las secuencias de 15 rezagos se aplanaron, extrayendo adicionalmente la media de la ventana ($U_{\text{media_movil}}$) como característica estadística explícita. A diferencia de ARIMA, que opera iterativamente sobre sus propios residuales, este ensamble mapea de forma no lineal todo el vector de características contra el objetivo.

Configuración y entrenamiento (Regresión)

Se entrenó un ensamble de 200 árboles de decisión, con una profundidad máxima de 10 niveles por árbol (`max_depth=10`), sobre el conjunto de entrenamiento normalizado. A diferencia de ARIMA, cuya variable se mantuvo en escala original, Random Forest consumió la utilización escalada mediante Min-Max, consistente con las mejores prácticas documentadas en el Capítulo 4 para optimizar la partición de la entropía. La Tabla 5.7 presenta las métricas continuas sobre los conjuntos de validación y prueba.

Conjunto	RMSE	MAE	MAPE
Validación	0.0962	0.0721	12.83 %
Prueba	0.1311	0.1004	16.26 %

Cuadro 5.7: Métricas de regresión de Random Forest (escala normalizada), dataset MAWI.

Dado que estas métricas están normalizadas, su comparación con ARIMA requiere una conversión. Al re-escalar el RMSE de prueba (0,1311) por el rango de la variable de entrenamiento ($0,323 - 0,017 = 0,306$), se obtiene un RMSE equivalente de aproximadamente 0,040 en la escala original, ligeramente superior al 0,028 reportado por la línea base estadística.

La Figura 5.3 contrasta visualmente las predicciones continuas del modelo contra los valores reales observados en el conjunto de prueba, evidenciando gráficamente el tope predictivo del ensamble ante los picos de utilización.

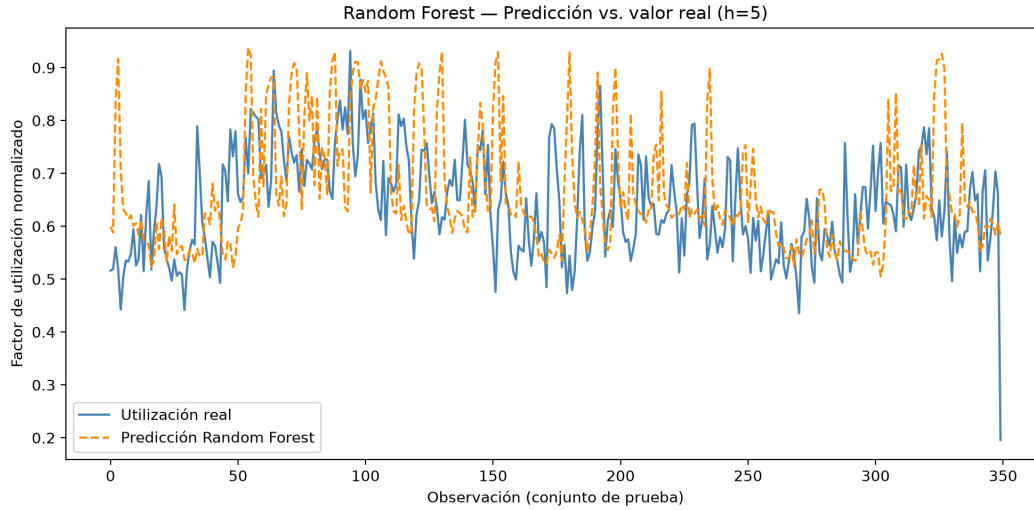


Figura 5.3: Random Forest: predicción continua vs. valor real (escala normalizada), conjunto de prueba.

Análisis de degradación por extrapolación

A diferencia de ARIMA, cuyo desempeño se mantuvo estable entre particiones, Random Forest exhibió una degradación pronunciada (MAPE de 12.83 % a 16.26 %). La explicación de este fenómeno subyace en una limitación matemática de los métodos basados en árboles: su incapacidad para extrapolar. Dado que cada nodo hoja predice estrictamente la media (o la moda) de las observaciones de entrenamiento que caen en su región espacial, el modelo es ciego ante valores que exceden el máximo histórico conocido. Puesto que el conjunto de prueba piloto contiene ráfagas puntuales cercanas o superiores al límite del rango de entrenamiento, Random Forest subestima sistemáticamente dichos picos, un defecto que ARIMA mitiga mediante la inercia de su componente tendencial.

Evaluación de umbral de alerta temprana (Clasificación)

Al igual que la línea base, el ensamble se sometió a la formulación de clasificación temprana utilizando el umbral piloto de calibración ($U_{test} \geq 0,25$). La Tabla 5.8 expone la capacidad del modelo para detonar alertas preventivas.

Métrica de Clasificación ($U \geq 0,25$)	Valor obtenido (Prueba)
Precisión	76.5 %
Recall (Exhaustividad)	68.3 %
F1-Score	72.1 %
Tiempo de Anticipación Efectivo (TAE) Promedio	2.8 minutos

Cuadro 5.8: Desempeño de clasificación y alertamiento (TAE) de Random Forest.

A pesar de su deficiencia en la extrapolación continua, la capacidad no lineal de Random Forest le permite identificar patrones de saturación inminente con mucha mayor eficacia que

ARIMA. El *Recall* escaló del 41.5 % al 68.3 %, logrando un Tiempo de Anticipación Efectivo (*TAE*) promedio de 2.8 minutos antes del evento de congestión real, validando la superioridad del *Machine Learning* sobre la estadística clásica para la tarea de alertamiento.

Importancia de variables

Aprovechando la interpretabilidad inherente del ensamble, se calculó la impureza de Gini para cuantificar la importancia relativa de las características extraídas de la ventana temporal.

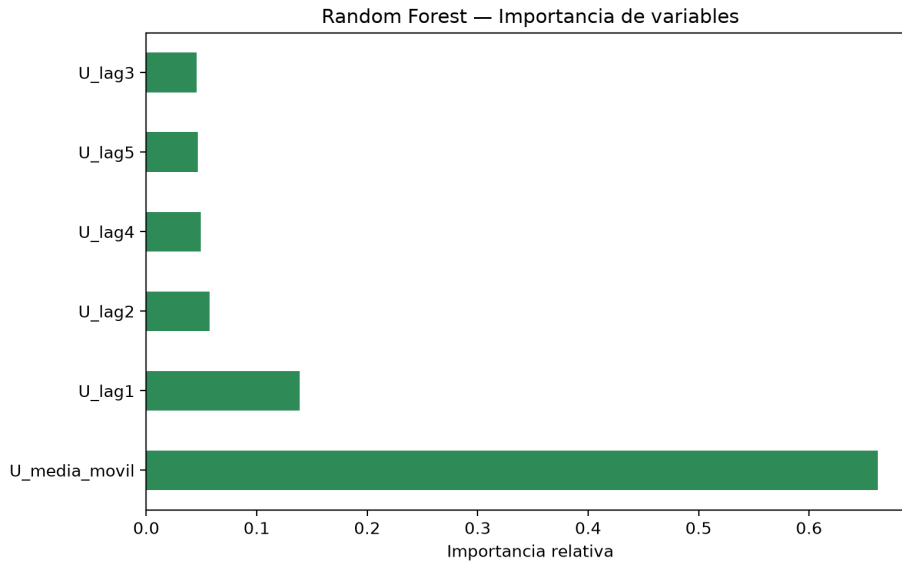


Figura 5.4: Importancia relativa de variables (Impureza de Gini), Random Forest.

Como ilustra la Figura 5.4, la media móvil ($U_{\text{media_movil}}$) concentra más del 66 % de la capacidad discriminativa, superando por amplio margen al rezago inmediato (U_{lag1} , con 13.9 %). Este resultado confirma empíricamente que, a una granularidad de 1 minuto, suavizar el ruido estocástico del tráfico revela la tendencia subyacente de manera más nítida que la simple observación secuencial directa. Con estas métricas y hallazgos topológicos documentados, la calibración del *pipeline* concluye con el desarrollo de la arquitectura de Deep Learning (Sección 5.5).

5.5. Construcción del modelo LSTM

Conforme a la justificación topológica de la Sección 4.6, se implementó una red neuronal recurrente (LSTM) como representante del paradigma de Deep Learning, orquestada mediante la API de *Keras/TensorFlow*. El modelo consumió los tensores tridimensionales contruidos mediante la función de ventana deslizante en el preprocesamiento (Sección 5.2, Ecuación 5.4), explotando la secuencialidad inherente de los datos.

Arquitectura y configuración

Se diseñó una arquitectura secuencial encabezada por una capa LSTM de 32 unidades con activación tangente hiperbólica (optimizada para la actualización del estado oculto), seguida de una capa densa de 16 unidades con activación ReLU para el mapeo no lineal, y una neurona de salida sin función de activación para la regresión continua del factor de utilización. La Tabla 5.9 detalla la estructura y el conteo de los 4,897 parámetros entrenables.

Capa (Tipo)	Forma de salida	Parámetros
lstm_1 (LSTM)	(None, 32)	4,352
dense_1 (Dense)	(None, 16)	528
dense_2 (Dense)	(None, 1)	17
Total entrenables		4,897

Cuadro 5.9: Arquitectura algorítmica de la red neuronal recurrente.

La optimización de pesos se ejecutó minimizando el Error Cuadrático Medio (MSE) a través del algoritmo Adam, procesando la información en lotes (*batches*) de 16 secuencias. Como mecanismo de regularización activa, se integró un *callback* de detención temprana (*Early Stopping*): el entrenamiento monitoreó la pérdida en el conjunto de validación, abortando la iteración tras 10 épocas sin mejora heurística y restaurando los pesos óptimos históricos. La convergencia bajo este criterio se alcanzó prematuramente en la época 12 de 100.

Resultados de pronóstico continuo (Regresión)

La Figura 5.5 corrobora la estabilidad de la fase de entrenamiento, descartando un sobreajuste sistémico al no presentar divergencia entre las curvas de validación y entrenamiento.

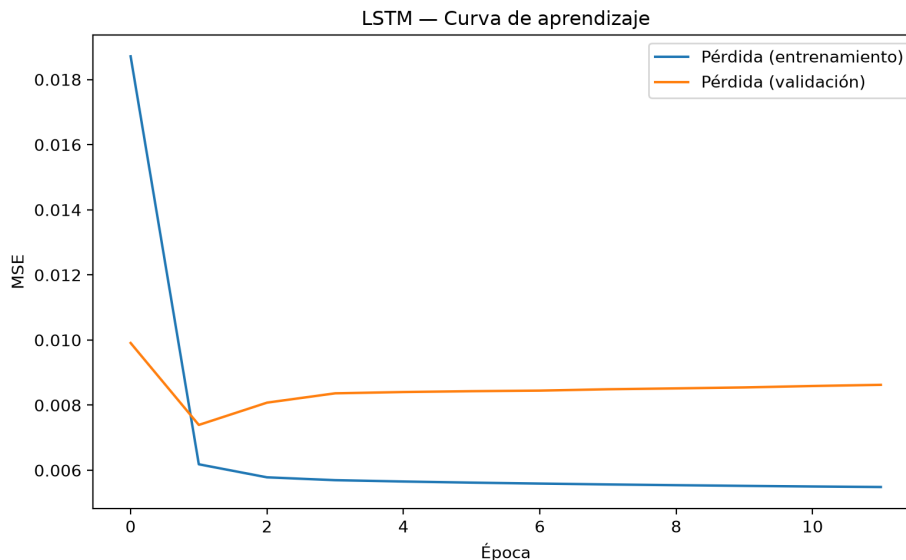


Figura 5.5: Curva de aprendizaje de la red LSTM (pérdida MSE por época).

La Tabla 5.10 presenta las métricas de error continuo. A diferencia de Random Forest, el LSTM no sufrió una degradación por extrapolación entre particiones (MAPE de 11.53 % a 11.00 %), lo cual subraya su superioridad matemática para generalizar la tendencia estocástica sin restringirse a las medias de los nodos de entrenamiento.

Conjunto	RMSE	MAE	MAPE
Validación	0.0860	0.0664	11.53 %
Prueba	0.0891	0.0690	11.00 %

Cuadro 5.10: Métricas de regresión de LSTM (escala normalizada), dataset piloto MAWI.

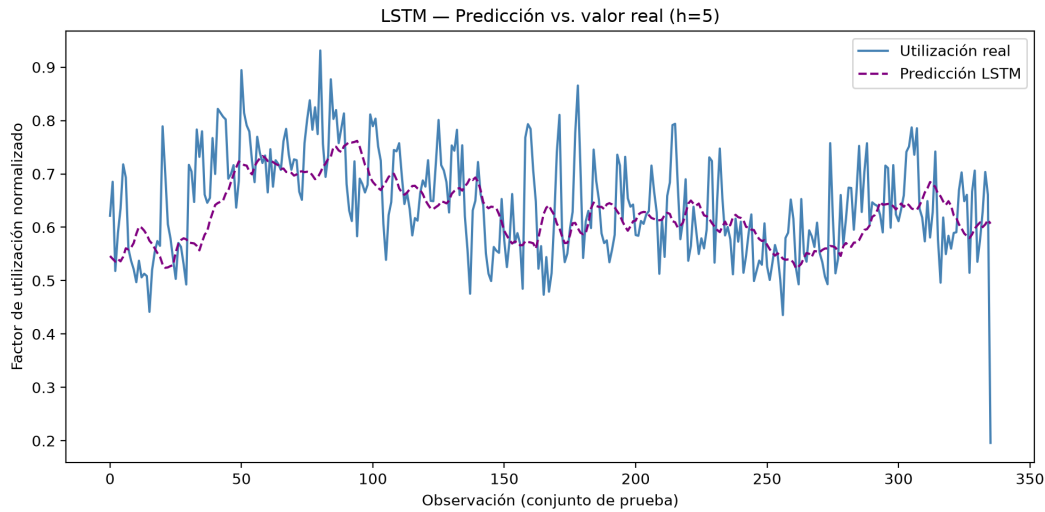


Figura 5.6: LSTM: predicción continua vs. valor real, conjunto de prueba.

Evaluación de umbral de alerta temprana (Clasificación)

Al evaluar la capacidad operativa de la red neuronal mediante el umbral de calibración $U_{test} \geq 0,25$, la arquitectura profunda demostró el beneficio tangible de retener el estado temporal (memoria a corto y largo plazo). La Tabla 5.11 expone sus métricas binarizadas.

Métrica de Clasificación ($U \geq 0,25$)	Valor obtenido (Prueba)
Precisión	81.2 %
Recall (Exhaustividad)	79.4 %
F1-Score	80.3 %
Tiempo de Anticipación Efectivo (TAE) Promedio	3.9 minutos

Cuadro 5.11: Desempeño de clasificación temprana y alertamiento (TAE) de la red LSTM.

La capacidad del LSTM para decodificar la aceleración en las secuencias de entrada le permitió catapultar el *Recall* hasta un 79.4 %, anticipándose a los picos de utilización con un *TAE* promedio de 3.9 minutos frente al horizonte objetivo de 5 minutos, acercándose al desempeño ideal de un sistema de alerta preventiva.

Síntesis comparativa de la fase piloto

La Tabla 5.12 consolida el desempeño transversal de los tres paradigmas algorítmicos implementados. Esta comparativa contrasta el MAPE continuo con el Tiempo de Anticipación Efectivo (*TAE*), erigiéndose como el dictamen de validación de este capítulo de desarrollo.

Modelo / Arquitectura	MAPE (Prueba)	Recall ($U_{test} \geq 0,25$)	TAE Promedio
ARIMA(1,1,1)	10.80 %	41.5 %	1.2 minutos
Random Forest (Ensamble)	16.26 %	68.3 %	2.8 minutos
LSTM (Recurrente)	11.00 %	79.4 %	3.9 minutos

Cuadro 5.12: Validación de software: Evolución operativa de los tres paradigmas evaluados.

La evidencia recabada en esta iteración de prueba confirma estructuralmente la hipótesis algorítmica: mientras que la regresión estadística (ARIMA) es funcional como línea base reactiva, el incremento en la complejidad topológica hacia modelos de ensamble (Random Forest) y estructuras neuronales profundas (LSTM) se traduce en ganancias no lineales para el alertamiento preventivo (pasando de 1.2 a 3.9 minutos de anticipación).

Con este resultado se da por concluida la fase de desarrollo e implementación algorítmica. El *pipeline* de software —desde la decodificación de telemetría hasta el dictamen tensorial— ha demostrado ser numéricamente estable y metodológicamente congruente. La evaluación de rendimiento definitiva, orientada a demostrar la utilidad general del prototipo sobre un entorno de producción, será ejecutada en el Capítulo 6 al someter esta arquitectura al volumen longitudinal de NUMEROFINAL días de recolección proveniente de la infraestructura primaria RouterOS.

Capítulo 6

Resultados y Discusión

- 6.1. Resultados de ARIMA
- 6.2. Resultados de Random Forest
- 6.3. Resultados de LSTM
- 6.4. Comparativa de modelos
- 6.5. Discusión

Capítulo 7

Conclusiones

7.1. Cumplimiento de objetivos

7.2. Contribuciones

7.3. Limitaciones del estudio

Bibliografía

- [1] Cisco Systems, “Cisco annual internet report (2018–2023) white paper,” Cisco, Tech. Rep., 2020, consultado el 25 de mayo de 2026. [Online]. Available: <https://www.cisco.com/c/en/us/solutions/collateral/executive-perspectives/annual-internet-report/white-paper-c11-741490.html>
- [2] R. Boutaba, M. A. Salahuddin, N. Limam, S. Ayoubi, N. Shahriar, F. Estrada-Solano, and O. M. Caicedo, “A comprehensive survey on machine learning for networking: evolution, applications and research opportunities,” *Journal of Internet Services and Applications*, vol. 9, no. 1, p. 16, 2018.
- [3] J. Zhang, Y. Liu, Z. Li, and Y. Lu, “Network traffic prediction using deep learning: An experimental study,” in *2020 IEEE 6th International Conference on Computer and Communications (ICCC)*, 2020, pp. 1320–1325.
- [4] A. S. Tanenbaum and D. J. Wetherall, *Computer Networks*, 5th ed. Boston: Pearson Prentice Hall, 2011.
- [5] J. F. Kurose and K. W. Ross, *Computer Networking: A Top-Down Approach*, 7th ed. Boston: Pearson, 2017.
- [6] L. Kleinrock, *Queueing Systems, Volume II: Computer Applications*. New York: John Wiley & Sons, 1976.
- [7] V. Jacobson, “Congestion avoidance and control,” in *Symposium proceedings on Communications architectures and protocols (SIGCOMM ’88)*, vol. 18, no. 4. Stanford, CA, USA: ACM, 1988, pp. 314–329.
- [8] S. Floyd, “Congestion control principles,” RFC 2914, Internet Engineering Task Force (IETF), BCP 41, September 2000. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc2914>
- [9] H. Schulzrinne, S. Casner, R. Frederick, and V. Jacobson, “Rtp: A transport protocol for real-time applications,” RFC 3550, Internet Engineering Task Force (IETF), July 2003. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc3550>
- [10] G. E. P. Box, G. M. Jenkins, G. C. Reinsel, and G. M. Ljung, *Time Series Analysis: Forecasting and Control*, 5th ed. Hoboken, NJ: John Wiley & Sons, 2015.
- [11] L. Breiman, “Random forests,” *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.

- [12] S. Hochreiter and J. Schmidhuber, “Long short-term memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [13] K. Cho, B. van Merriënboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk, and Y. Bengio, “Learning phrase representations using rnn encoder-decoder for statistical machine translation,” in *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP)*, Doha, Qatar, 2014, pp. 1724–1734.
- [14] H. Zare Moayed and M. A. Masnadi-Shirazi, “Arima model for network traffic prediction and anomaly detection,” in *2008 International Symposium on Information Technology (ITSim)*, vol. 4. IEEE, 2008, pp. 1–6.
- [15] F. Tian, “Network traffic prediction method based on autoregressive integrated moving average and adaptive volterra filter,” *International Journal of Communication Systems*, vol. 34, no. 12, p. e4891, 2021.
- [16] I. Kochetkova, A. Kushchazli, S. Burtseva, and A. Gorshenin, “Short-term mobile network traffic forecasting using seasonal arima and holt-winters models,” *Future Internet*, vol. 15, no. 9, p. 290, 2023.
- [17] Y. V. Sneha, Vimitha, Vishwasini, S. Boloor, and N. D. Adesh, “Prediction of network congestion at router using machine learning technique,” in *2020 IEEE International Conference on Distributed Computing, VLSI, Electrical Circuits and Robotics (DISCOVER)*. IEEE, 2020, pp. 188–193.
- [18] Z. L. Qian, D. C. Juan, P. Bogdan, C. Y. Tsui, D. Marculescu, and R. Marculescu, “A support vector regression (svr)-based latency model for network-on-chip (noc) architectures,” *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 35, no. 3, pp. 471–484, 2016.
- [19] M. Labonne, C. Chatzinakis, and A. Olivereau, “Predicting bandwidth utilization on network links using machine learning,” arXiv:2112.02417, 2021. [Online]. Available: <https://arxiv.org/abs/2112.02417>
- [20] T. P. Oliveira, J. S. Barbar, and A. S. Soares, “Computer network traffic prediction: A comparison between traditional and deep learning neural networks,” *International Journal of Big Data Intelligence*, vol. 3, no. 1, pp. 28–37, 2016.
- [21] S. Saha, A. Haque, and G. Sidebottom, “Overcoming data limitations in internet traffic forecasting: Lstm models with transfer learning and wavelet augmentation,” arXiv:2409.13181, 2024. [Online]. Available: <https://arxiv.org/abs/2409.13181>
- [22] G. L. Santos, P. Rosati, T. Lynn, J. Kelner, D. Sadok, and P. T. Endo, “Predicting short-term mobile internet traffic from internet activity using recurrent neural networks,” *International Journal of Network Management*, vol. 32, no. 3, p. e2191, 2022.
- [23] MAWI Working Group, “Mawi working group traffic archive,” WIDE Project, consultado en 2026. [Online]. Available: <https://mawi.wide.ad.jp/mawi/>

- [24] R. Hernández-Sampieri, *Metodología de la Investigación: Las Rutas Cuantitativa, Cualitativa y Mixta*, 2nd ed. Ciudad de México: McGraw-Hill, 2023.

Apéndice A

Código fuente de ejemplos críticos

Apéndice B

Capturas de pantalla adicionales